Adaptive Sorting, Also in the BST Model

Ivor van der Hoog
IT University of Copenhagen, Denmark

Eva Rotenberg
IT University of Copenhagen, Denmark

Daniel Rutschmann
Institute of Science and Technology Austria (ISTA), Austria

Abstract

Sorting is one of the most basic algorithmic problems. In adaptive sorting, algorithms perform better on inputs that are easier to sort. Formally, one defines a measure of sortedness that captures features of the input, typically an entropy measure, and tries to design algorithms that adapt to this measure. Over the years, many such measures have been studied, raising two natural questions: How do these measures relate to each other, and are there algorithms that are adaptive to each measure?

We answer these questions by completing a hierarchy of relations between previously studied entropy measures. Our added relations show that this hierarchy has a source, Reg, which is a classical measure from the literature. We then ***discover*** and *prove* that an existing data structures already yields a sorting algorithm adaptive to Reg. Consequently, this algorithm already achieve the adaptive bounds of all other measures in the hierarchy.

We then make a brief mathematical detour. McIlroy previously noted that the classical measure Reg has a mathematically inconvenient definition. We argue for studying instead a slightly weaker, but mathematically much more elegant measure reg. We show that $O(n + \log \mathrm{reg}) = O(n + \log Reg)$, and that reg is more elegant and easier to analyse.

Finally, we turn to algorithm design. We introduce SproutSort, a reg-adaptive algorithm in the offline binary search tree model (the *BST Model*). In this model, a single binary search tree stores the distinct input keys, which must be accessed in input order. The entire access sequence is known in advance, but each access starts at the root and incurs unit cost for pointer moves and rotations. SproutSort achieves an access cost of $O(n + \log \mathrm{reg})$, making it the most adaptive algorithm in the BST Model to date. Turning our attention back to adaptive sorting in the comparison model, we also present the adaptive sorting algorithm SkeletonSort. This is a simple randomised pointer-machine algorithm that sorts distinct keys in expected $O(n + \log \mathrm{reg}) = O(n = \log \mathrm{Reg})$ time. Although it does not work if the input has duplicates, SkeletonSort offers a considerably simpler implementation of an algorithm this high in the adaptive hierarchy.

2012 ACM Subject Classification Theory of computation → Design and analysis of algorithms; Theory of computation → Sorting and searching

Keywords and phrases Adaptive sorting, entropy, algorithms

Digital Object Identifier 10.4230/LIPIcs...

Funding *Ivor van der Hoog*, *Eva Rotenberg*, and *Daniel Rutschmann* are grateful to the VILLUM Foundation for supporting this work via Eva Rotenberg's grant (VIL37507) "Efficient Recomputations for Changeful Problems". This work was supported by the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 899987.

© Ivor van der Hoog, Eva Rotenberg, and Daniel Rutschmann;
licensed under Creative Commons License CC-BY 4.0

1 Introduction

This paper studies comparison-based adaptive sorting of n elements and the related offline binary search tree (BST) access problem. Sorting n distinct elements requires $\Omega(n \log n)$ comparisons in the worst case. However, this lower bound does not take into account the fact that some inputs are easier to solve than others. If the input is already sorted, one can verify this in linear time and, intuitively, an *almost sorted input* can also be sorted faster. A large body of work studies *adaptive sorting algorithms* that exploit input features that reflect notions of almost-sortedness, expressed by entropy functions. Examples include inputs consisting of few sorted contiguous pieces [24, 27], few contiguous blocks [10], or few (possibly interleaved) increasing or monotone subsequences [26]. Whenever a measure of entropy is introduced, a natural question is how it relates to other measures: is one strictly stronger than the other, in the sense that matching one in running time, implies matching the other? In this paper, we extend the hierarchy of entropy measures by establishing the implications marked in orange in Figure 1. As a consequence, the hierarchy has a source Reg.

For each entropy measure in the hierarchy, one can ask whether there exists algorithms that are adaptive to that measure. We ***discover*** that the unified bound dictionary of Bădoiu, Cole, Demaine, and Iacono [3] already yields a Reg-adaptive algorithm. Thus, [3] yields the most adaptive sorting algorithms to date. We continue by modifying Reg to define a simpler measure reg with much nicer mathematical properties. We show that reg dominates almost all prior measures. Moreover, if the input contains no duplicates then we prove that any reg-adaptive algorithm is also Reg-adaptive. We give a reg-adaptive offline BST access algorithm. Finally, we present a simple reg-adaptive classical algorithm.

Terminology. To explain our results, consider the following abstractions of adaptive sorting bounds: let $\mathcal{I}_n$ denote the set of all size-n inputs. A *notion of presortedness* assigns to every input $X \in \mathcal{I}_n$ some target cost $f(X)$. A sorting algorithm is *comparison-adaptive* with respect to this notion if it sorts every input X in $O(f(X))$ comparisons, and *time-adaptive* if its total running time is $O(f(X))$. For offline BST algorithms, we consider the BST model where one can navigate and rotate nodes of a binary search tree, and we consider the total access cost of accessing all elements of X in-order. The target f is usually defined in two steps: first a number $k(X)$ is defined in a sophisticated way, and then $f(X)$ is a function of n and $k(X)$, e.g. $n(1+\log k(X))$ or $n(1+k(X))$. A concrete example is the number of runs [24]: By splitting X between every pair of adjacent items that form a decreasing sequence of length 2, we partition X into maximal contiguous non-decreasing sequences $X_1, X_2, \ldots, X_{k=\text{nRuns}}$. Then, [24] puts $f(X) = n(1 + \log(\text{nRuns}))$ and presents a (comparison) adaptive algorithm.

Ranks and Duality. For a list X of elements and $x \in X$ we define the rank of x denoted by $\text{rank}_X(x)$ as its index in the sorted order, *after* removing duplicates. Consider for the sake of analysis, simultaneously replacing each element $X[i]$ by $\text{rank}_X(X[i])$. Then any comparison-based algorithm makes the exact same decisions for the replaced input. Thus, we may assume that X is a list of elements in $[n]$. If X contains no duplicates then X is a permutation. If X is a permutation, it has a unique inverse X^{-1} which we call its dual, and which leads to dual notions of presortedness. For example, dual runs: $\text{nRuns}^*(X) := \text{nRuns}(X^{-1})$, giving rise to $f(X) = n(1 + \log(\text{nRuns}^*(X)))$. In most cases, these dual definitions can be (manually) extended to inputs X with duplicates. E.g., A subsequence $X_i \subseteq X$ is *value-contiguous* if for all $x, y, z \in X$ with $x \leq y \leq z$ and $x, z \in X_i$, we also have $y \in X_i$. We can partition $X = X_1 \sqcup \cdots \sqcup X_k$ into maximal non-decreasing value-contiguous sequences X_i and put

$\mathrm{nRuns}^*(X) := k$; this agrees with the previous definition when X contains no duplicates. We *occasionally* define duality for lists with duplicates, by considering the position of the i'th element in the sorted order after a stable sort, denoted $\overline{X}[i]$. Note that $\overline{X}$ is a permutation.

Entropy functions. The Shannon entropy function allows us to define strong notions of presortedness from a given partition of X. We illustrate three examples: For runs, consider the partition $X = X_1 \sqcup \cdots \sqcup X_k$ into maximal contiguous non-decreasing sequences, put $\mathrm{HRuns} := \sum_{i=1}^{k} \frac{|X_i|}{n} \log_2 \frac{n}{|X_i|}$ and let $f_{\mathrm{HRuns}}(X) = n(1+\mathrm{HRuns}(X))$. For dual runs, consider the partition $X = X_1 \sqcup \cdots \sqcup X_k$ into maximal *value-contiguous* non-decreasing sequences, put $\mathrm{HRuns}^* := \sum_{i=1}^{k} \frac{|X_i|}{n} \log_2 \frac{n}{|X_i|}$ and let $f_{\mathrm{HRuns}^*}(X) = n(1+\mathrm{HRuns}^*(X))$. McIlroy [28] shows that merge sort with exponential search is both HRuns-adaptive and HRuns^*-adaptive. For multi-set sorting, assume $X \subseteq [n]^n$ and denote for $i \in [n]$ let m_i be the number of occurrences of i. Put $\mathrm{HMulti} := \sum_{i=1}^{n} \frac{m_i}{n} \log \frac{n}{m_i}$ and $f_{\mathrm{HMulti}}(X) := n(1+\mathrm{HMulti}(X))$. [30] shows that merge sort, with a minor adaption, is HMulti-adaptive.

A hierarchy. Measures of presortedness form a hierarchy. We draw an arrow from a measure M to a measure N, and write $M \preceq N$, if $f_M(X) \in O(f_N(X))$ for all inputs X. In other words, any M-adaptive algorithm is automatically also N-adaptive. For example, all occurrences of the same value in X form a value-contiguous subsequence of X, and each dual run is a union of such sequences. Consequently, $\mathrm{HRuns}^*(X) \leq \mathrm{HMulti}(X)$, so an algorithm being HRuns^*-adaptive implies adaptivity for multi-set sorting. Figure 1 illustrates the hierarchy of the measures considered here. Its source is Reg. Here, we explain the definition of Reg which is complicated, and considers for each element i its rank in $X[1..i]$.

$$\mathrm{Reg}(X) := \prod_{i=2}^{n} \min_{j<i} \left\{ (i-j) \cdot \max\left\{1, \left|\mathrm{rank}_{X[1..i]}(X[i]) - \mathrm{rank}_{X[1..i]}(X[j])\right|\right\}\right\}.$$

The measure Reg is derived from insertion sort: process X from left to right and insert $X[i]$ into $X[1..(i-1)]$ using a good *finger* $X[j]$. A good finger is close to $X[i]$ both in terms of index and rank. Crucially, and inconveniently (as acknowledged by McIlroy [28]), these ranks are taken only among the elements seen so far. For example, for $X = (1,5,4,3,2)$ consider $i = 2, j = 1$: when $X[2] = 5$ is inserted, only $X[1] = 1$ and $X[2] = 5$ have been seen, so the rank difference is 1 and the cost is $(2-1)|2-1| = 1$, even though $|X[2] - X[1]| = 4$. Thus, Reg considers this insertion to be cheap as $2, 3, 4$ have not yet been inserted. Unfortunately, this makes Reg not invariant under reversing X. Our alternative, weaker, but more natural definition reg simply considers for $X[i]$ and $X[j]$ their ranks in the full input X; the target cost $n + \log \mathrm{reg}(X)$ is invariant up to constant factors under reversing the sequence or its value order and, for permutations, under inversion: $n + \log \mathrm{reg}(X) \in \Theta(n + \log \mathrm{reg}(X^{-1}))$.

$$\mathrm{reg}(X) := \prod_{i=2}^{n} \min_{j<i} \left\{ (i-j) \cdot \max\left\{1, \left|\mathrm{rank}_{X}(X[i]) - \mathrm{rank}_{X}(X[j])\right|\right\}\right\}.$$

Problem statements. Our input is a list X of n elements from a totally ordered universe, which may contain duplicates. The first is comparison-based sorting: compute a non-decreasing ordering of X using only comparisons to determine the relative order of elements. The second is offline BST access: given a BST containing exactly the distinct values of X, access $X[1], X[2], \ldots, X[n]$ in input order (see Section 3 for the formal definition of the BST model). The goal is to obtain adaptive bounds for both problems.

Contributions. Our first contribution, is that we establish the hierarchy in Figure 1, adding several new or previously unpublished relations, establishing that Reg is its source. Moffat and Petersson [29] achieved an algorithm that used Reg-adaptive comparisons, but whose running time has an additive $O(n \log\log n)$ time overhead. We ***discover*** that insertion sort using the unified bound dictionary of Bădoiu, Cole, Demaine, and Iacono [3] takes $O(n + \log \mathrm{Reg}(X))$ time, eliminating this overhead. We also show that heapsort using the unified bound heap of Rutschmann [32] takes $O(n + \log \mathrm{reg}(X))$ time (we could make a stronger statement by expanding the concept of duality, but refrain from doing so). We then study the mathematically more convenient measure reg:

This measure measures rank distances in the *full input*. We show an interesting mathematical property that $n + \log \mathrm{reg}(X)$ is, up to constant factors, the weight of a minimum spanning tree with edge weights $\log(2 + |i - j| + |\operatorname{rank}_X(|X[i]) - \operatorname{rank}(X[j])|)$. This characterisation explains the symmetric properties of reg. Moreover, reg dominates every measure in Figure 1 except the measures that consider prefix ranks Reg, Hist, and Loc. Finally, we show that if X contains no duplicates then $O(n + \log \mathrm{reg}) = O(n + \log \mathrm{Reg})$ (Theorem 11).

Our first new algorithm, SproutSort, addresses offline BST access. Given the rank of each element and its multiplicity, it implements a BST merge sort of cost $O(n + \log \mathrm{reg}(X))$. Through the framework of Blelloch and Dobson [8] and our extension to repeated keys, it yields a deterministic offline BST access algorithm with the same cost. Existing unified bound dictionaries and heaps lie outside the BST model, while known BST algorithms incur an additive $O(n \log\log n)$ overhead [16, 17]. The Reg-adaptive algorithms can first compute the ranks and multiplicities [3], and so our access algorithm runs in $O(n + \log \mathrm{reg}(X))$ time.

Finally, we present SkeletonSort, a simple randomised pointer-machine sorting algorithm. It sorts distinct comparable keys in expected $O(n + \log \mathrm{reg}(X))$ time without prior knowledge of their ranks. Since $O(n + \log \mathrm{reg}(X)) = O(n + \log \mathrm{Reg}(X))$, this algorithm is as good as [3] when the input has no duplicates.

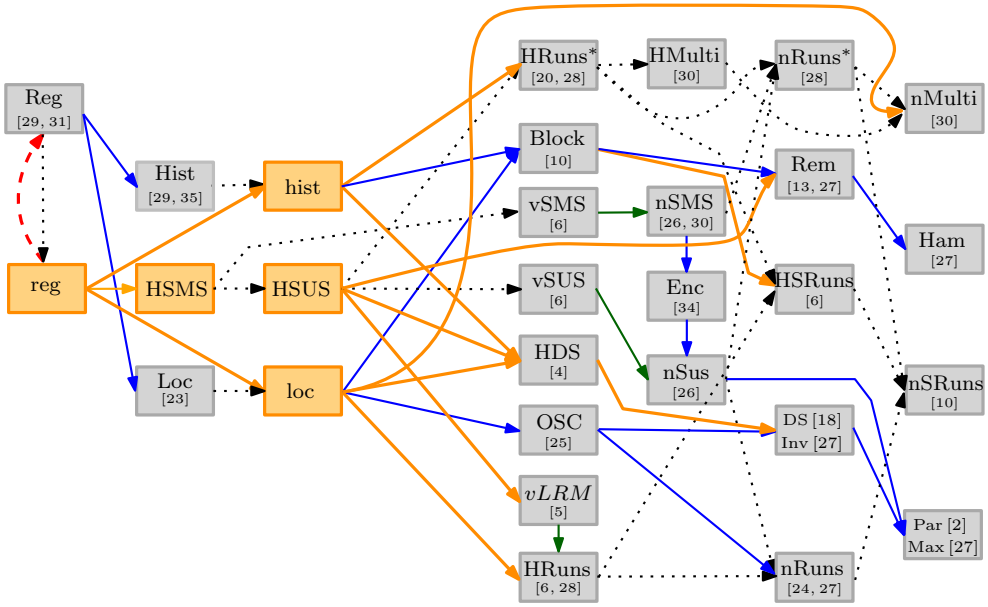


Figure 1 The hierarchy. Orange&red is established in this paper, red only holds for permutations. Blue is from [31], green from [6], and black is straightforward. For entropy measure definitions, see Table 1. We move the base of the blue arrows [31] from their original Hist and Loc to hist and loc, although these were not defined at the time of [31], since it follows from the essence of their proof.

2 Discovery

We discover that Reg corresponds to the unified bound for dictionaries [3]. We expand in Appendix C the hierarchy of Figure 1 so that Reg dominates all other previous measures in the hierarchy. Thus, we *discover* that the dictionary from [3] yields an insertion-sort algorithm that dominates all previously studied adaptive sorting algorithms from [5–8, 10, 23, 25–30, 34].

We then revisit the observation of McIlroy [28] that Reg is mathematically inelegant. We make the case for a mathematically more convenient and elegant (but slightly weaker) measure reg, and show that this measure also suffices to dominate all measures in the hierarchy except for six equally mathematically inconvenient measures listed in Section 2.3. In Appendix D, we further explore the definitions of presortedness measures and explain how the classical Hist and Loc from Figure 1 correspond to the working set and dynamic finger properties for binary search trees, respectively. For completeness, Appendix E discusses the measure by Barbay, Ochoa and Satti [7].

Discovering a Reg**-adaptive algorithm.** Badoiu, Cole, Demaine and Iacono [3, Theorem 3] design a *unified bound* dictionary. We show that sorting with their dictionary is Reg-adaptive.

▶ **Definition 1.** *A dictionary maintains a set Y of distinct elements with multiplicities. Inserting x adds it to Y if absent, and otherwise increases its multiplicity. The dictionary can report Y in sorted order in $O(|Y|)$ time. Insertion sort inserts all elements of X, and then reports Y in sorted order, repeating each element according to its multiplicity. Let* $\mathrm{age}(y)$ *count the distinct elements accessed since the last access to y. The first insertion takes $O(1)$ time. Thereafter, the* unified bound *demands that inserting x takes amortized $O\left(\min_{y\in Y}\left\{\log(2+|\operatorname{rank}_{Y\cup\{x\}}(x)-\operatorname{rank}_{Y\cup\{x\}}(y)|)+\log(2+\mathrm{age}(y))\right\}\right)$ time.*

▶ **Theorem 2.** *Insertion sort with a unified bound dictionary sorts a sequence X of n elements in $O(n+\log \mathrm{Reg}(X))$ time and comparisons.*

Proof. We insert X into a unified bound dictionary. Processing $X[i]$ consists of inserting it if absent, or searching for it and increasing its multiplicity otherwise.

For every $j<i$, the last access to $X[j]$ occurred at or after step j, so $\mathrm{age}(X[j])\le i-j$. Since rank ignores duplicates, and $Y\cup\{X[i]\}$ and $X[1..i]$ contain the same values, we have $\operatorname{rank}_{Y\cup\{X[i]\}}(\cdot)=\operatorname{rank}_{X[1..i]}(\cdot)$. Thus, the unified bound gives a total running time of

$$O(n+\sum_{i=2}^{n}\min_{j<i}\left\{\log(2+|\operatorname{rank}_{X[1..i]}(X[i])-\operatorname{rank}_{X[1..i]}(X[j])|)+\log(2+i-j)\right\})=O(n+\log \mathrm{Reg}(X)).$$

Finally, we output the distinct elements in sorted order, repeating each according to its multiplicity, in $O(n)$ time. The number of comparisons is bounded by the running time. ◀

Discovering a ***-adaptive algorithm.** The measure Reg is odd: it measures rank distances in each prefix $X[1..i]$, and so it depends on the order in which elements are processed. Prefixes are natural for dictionaries, which insert elements from left to right. For heaps, however, we care about the elements *remaining* after each extraction! Rutschmann [32] introduced the unified bound for heaps. We show that, if the input X as no duplicates, then heapsort yields adaptivity to a different measure, *. This measure is obtained by taking the dual of Reg and replacing prefix ranks $\operatorname{rank}_{X[1..i]}$ with suffix ranks $\operatorname{rank}_{X[i..n]}$.

$$(X):=\prod_{i=2}^{n}\min_{j<i}\left\{(i-j)\cdot\max\left\{1,\left|\operatorname{rank}_{X[i..n]}(X[i])-\operatorname{rank}_{X[i..n]}(X[j])\right|\right\}\right\}$$

We define $^*(X) := (X^{-1})$ and recall unified heaps [32, Def. 25, ArXiV]:

▶ **Definition 3.** *A heap maintains a set Y of distinct elements. InsertAdds an element to Y, while ExtractMin removes and reports its minimum. Heap sort inserts all elements of X into the heap, and then repeatedly extracts the minimum.*

When inserting a sequence X, the unified bound *demands that Insert takes amortized $O(1)$ time, the first ExtractMin operation takes $O(1)$ time, and each subsequent operation extracting $X[i]$ takes amortized time*

$$O\left(\min_{X[j]<X[i]} \{\log(2+|X[i]-X[j]|) + \log(2+|\{k \in [\min(i,j), \max(i,j)] : X[k] \geq X[i]\}|)\}\right)$$

▶ **Theorem 4.** *Heap sort with a unified bound heap sorts a sequence X of n distinct elements in $O(n + \log^*(X))$ time and comparisons.*

Proof. We may assume that X is a permutation of $[n]$. We insert all elements into the heap.

Put $P = X^{-1}$. For every $v > u$, immediately before extracting v, the elements remaining in the heap are exactly those with value at least v. Thus, the quantity $|\{k \in [\min(P[v], P[u]), \max(P[v], P[u])] : X[k] \geq v\}|$ differs by at most 1 from $|\operatorname{rank}_{P[v..n]}(P[v]) - \operatorname{rank}_{P[v..n]}(P[u])|$. Indeed, the elements of $P[v..n]$ are precisely the insertion indices of the elements remaining in the heap. Consequently, the unified bound gives a total running time bounded by $O(n)$ plus, asymptotically,

$$\sum_{v=2}^{n} \min_{u<v} \left(\log(2+v-u) + \log(2+|\operatorname{rank}_{P[v..n]}(P[v]) - \operatorname{rank}_{P[v..n]}(P[u])|)\right) = O(n + \log(P)).$$

Since $P = X^{-1}$, this is $O(n + \log^*(X))$. ◀

2.1 The Case for reg

The measure Reg has a mathematically inconvenient property: reversing X may significantly change its value. For example, let $n = k^2$, put $S = (1, k+1, \ldots, (k-1)k+1)$, and let $X = (S, \ldots, S, 1, 2, \ldots, n)$, with k copies of S. In X, consecutive items have prefix-rank distance at most 1, except at the start of each copy of S, where a previous occurrence is at distance k. Thus, $\operatorname{Reg}(X) \leq k^k = 2^{o(n)}$. However, reversing X places $(n, n-1, \ldots, 1)$ first. Each of the remaining n accesses then has rank distance at least $k-1$ from any different value, and index distance at least k from any previous occurrence of the same value. Hence $\operatorname{Reg}(\operatorname{Reverse}(X)) = k^{\Theta(n)} = 2^{\Theta(n \log n)}$.

Consequently, there are two different, incomparable variants of Reg, and similarly two variants of *. For this reason, we wish to make the case for the simpler measure

$$\operatorname{reg}(X) := \prod_{i=2}^{n} \min_{j<i} \{(i-j) \cdot \max\{1, |X[i]-X[j]|\}\}.$$

In fact, McIlroy [28] already notes that Reg is needlessly complicated compared to reg (although he incorrectly claims that $\log \operatorname{reg} \leq \log \operatorname{Reg} + O(n)$, see above counterexample). Although reg is weaker than all variants of Reg and , it appears to capture the essence of both measures. We establish another property that we believe illustrates the mathematical elegance of reg: the bound $n + \log \operatorname{reg}$ is equivalent to the weight of a minimum spanning tree in a natural graph. This characterisation makes $O(n + \log \operatorname{reg})$ invariant up to constant factors under reversing X, reversing its values, or replacing X by X^{-1}. We show:

▶ **Theorem 5.** *Given X of length $n \geq 2$, consider the complete graph $G(X)$ on $[n]$, with edge weights $w(\{i,j\}) = \log(2 + |i-j| + |X[i] - X[j]|)$. Let $\mathrm{unimst}(X)$ be the weight of a minimum spanning tree of $G(X)$. Then $n + \log \mathrm{reg}(X) \in \Theta(\mathrm{unimst}(X))$.*

▶ **Definition 6.** *A* finger function *is a map $f : [n] \setminus \{1\} \to [n]$ with $f(i) < i$. Recall that $\mathrm{reg}(X)$ minimises $\prod_{i=2}^{n}(i - f(i)) \max\{1, |X[i] - X[f(i)]|\}$ over all finger functions f.*

▶ **Lemma 7.** *Every forest with m edges contains a spanning subgraph H with at least $m/2$ edges, such that every connected component of H is a star.*

Proof. Root each tree arbitrarily and partition its edges according to whether the child endpoint has even or odd depth. In either part, every vertex is adjacent either only to its parent or only to its children. Thus, every connected component is a star. Since the two parts partition the m edges, one contains at least $m/2$ edges. ◀

Proof of Theorem 5. Every finger function f defines a spanning tree, since $f(i) < i$ for all $i > 1$. Moreover, $w(\{i, f(i)\})$ is at most $2 + \log(i - f(i)) + \log \max\{1, |X[i] - X[f(i)]|\}$. Taking the minimum over all finger functions gives $\mathrm{unimst}(X) \leq 2n + \log \mathrm{reg}(X)$.

Conversely, let T be a minimum spanning tree. We construct a finger function f whose edges have total weight $O(w(T) + n)$.

Let T' be a copy of T. Initially, every vertex forms its own 0-cluster. In rounds $r = 1, 2, \ldots$, contract all $(r-1)$-clusters and consider the forest F of edges whose current weight lies in $[2r-2, 2r)$. By Lemma 7, choose a star forest $H \subseteq F$ containing at least half its edges. Merge the clusters of each star into an r-cluster, and increase the weight of every edge in $F - H$ by 2. Clusters not involved in a merge remain unchanged.

We first bound the diameter of the clusters. Put $d(i,j) = 2^{w(i,j)}$. By the triangle inequality, $d(i,j) \leq d(i,k) + d(k,j)$. Let D_r be the maximum d-diameter of an r-cluster. Since every star has diameter realized by at most two edges, each of weight less than $2r$, we have $D_0 = 2$ and $D_r \leq 3D_{r-1} + 2 \cdot 2^{2r}$. Induction gives $D_r \leq 2^{2r+3}$. Thus, any two vertices in the same r-cluster have weight at most $2r + 3$.

The *leader* of a cluster is its smallest-index vertex. Whenever an r-cluster merges k smaller clusters with leaders $i_1 < \cdots < i_k$, we set $f(i_j) = i_1$ for every $j > 1$. This is valid since $i_1 < i_j$. Moreover, each vertex receives a finger exactly when it ceases to be a leader. Since eventually all vertices belong to one cluster, whose leader is 1, every vertex except 1 receives exactly one finger. These $k - 1$ finger edges have weight at most $2r + 3$, while the corresponding star contains $k - 1$ edges of weight at least $2r - 2$. Summing over all merges gives $w(f) \leq w(T') + 5(n-1)$.

Finally, every round uses at least as many edges as it increases in weight. Since each edge is used in exactly one merge, the total weight increase is at most $2(n-1)$. Hence $w(T') \leq w(T) + 2(n-1)$, and therefore $w(f) \in O(w(T) + n)$. Since $\log \mathrm{reg}(X) \leq 2w(f)$ and $w(T) \geq n - 1$, we conclude that $n + \log \mathrm{reg}(X) \in O(\mathrm{unimst}(X))$. ◀

2.2 Clean definitions for existing measures: loc, hist, HSMS, HSUS

In addition to Reg, Moffat and Petersson [29] define Hist and Loc (see Table 1), which lie high in the hierarchy of Figure 1. Both measures have been studied through adaptive sorting algorithms [8, 29]. Like Reg, they use prefix ranks and thus share its mathematical inconveniences. McIlroy [28] briefly mentioned more elegant alternatives, analogous to reg:

$$\mathrm{loc}(X) = \prod_{i=2}^{n} \max\{1, |X[i] - X[i-1]|\} \quad \text{and} \quad \mathrm{hist}(X) = \prod_{i=2}^{n} |\overline{X}^{-1}[i] - \overline{X}^{-1}[i-1]|.$$

We can define mathematically convenient counterparts for more existing measures: Levcopoulos and Petersson [26] study partitions $X = X_1 \sqcup \cdots \sqcup X_k$ into non-decreasing subsequences. As the X_i need not be contiguous or value-contiguous, this generalises both runs and dual runs. They define nSUS as the minimum number k of parts, and give a sorting algorithm with adaptive bound $f_{\mathrm{nSUS}}(X) = n(1 + \log \mathrm{nSUS}(X))$. Barbay and Navarro [6] strengthen this bound by considering the entropy of the part sizes rather than their number. They compute a partition with nSUS parts and define $H(\mathrm{vSUS})$ as the entropy of its part sizes. Since any partition into k parts has entropy at most $\log k$, this strengthens the nSUS bound. However, as they observe, their partition need not minimize entropy. We therefore define

$$\mathrm{HSUS}(X) = \min_{X = X_1 \sqcup \cdots \sqcup X_k} \sum_{i=1}^{k} \frac{|X_i|}{n} \log \frac{n}{|X_i|}$$

where the minimum is over all partitions into non-decreasing subsequences. Clearly, $\mathrm{HSUS} \preceq H(\mathrm{vSUS})$, giving a stronger measure. We further generalize these partitions by allowing each X_i to be either non-decreasing or non-increasing. Such partitions are called *shuffled monotone sequences* (SMS). Levcopoulos and Petersson [26] define $\mathrm{nSMS}(X)$ as the minimum number of parts in such a partition. We define its entropy counterpart as

$$\mathrm{HSMS}(X) = \min_{X = X_1 \sqcup \cdots \sqcup X_k} \sum_{i=1}^{k} \frac{|X_i|}{n} \log \frac{n}{|X_i|}$$

where the minimum is over all partitions into monotone subsequences. Every SUS partition is also an SMS partition, so $\mathrm{HSMS} \preceq \mathrm{HSUS}$ and $\mathrm{nSMS} \preceq \mathrm{nSUS}$. Moreover, every partition into nSMS parts has entropy at most $\log \mathrm{nSMS}$, so $\mathrm{HSMS} \preceq \mathrm{nSMS}$. Our entropy-minimizing variants are therefore stronger than their classical counterparts, and independent of the choice of a particular partition. We further study these relations in Section 2.3 and Appendix C. Our main point is that reg dominates these strong, historically motivated measures.

2.3 The strength of reg

In Figure 1, reg dominates all measures except Reg, *, Hist, *, Loc, and *. These six measures all rely on ranks within a prefix or suffix of X, rather than within the full input. Replacing them by their full-rank counterparts reg, hist, and loc, leaves reg as the strongest measure. We illustrate this by proving $\mathrm{reg} \preceq \mathrm{loc}$, $\mathrm{reg} \preceq \mathrm{hist}$, and $\mathrm{reg} \preceq \mathrm{HSMS}$.

▶ **Lemma 8.** $\mathrm{reg} \preceq \mathrm{loc}$.

Proof. Choosing $f(i) = i - 1$ gives $\mathrm{reg}(X) \leq \prod_{i=2}^{n} \max\{1, |X[i] - X[i-1]|\} = \mathrm{loc}(X)$. ◀

▶ **Lemma 9.** $\mathrm{reg} \preceq \mathrm{hist}$.

Proof. First suppose that X contains no duplicates. By Lemma 8, $\mathrm{reg}(X^{-1}) \leq \mathrm{loc}(X^{-1}) = \mathrm{hist}(X)$. Moreover, recall that $G(X)$ is the complete graph on $[n]$ with edge weights $w(\{i,j\}) = \log(2 + |i-j| + |X[i] - X[j]|)$. Since X is a permutation, $G(X)$ and $G(X^{-1})$ are isomorphic: mapping each vertex i to $X[i]$ exchanges indices and values, preserving every edge weight. Thus, by Theorem 5, $n + \log \mathrm{reg}(X) \in \Theta(n + \log \mathrm{reg}(X^{-1})) \subseteq O(n + \log \mathrm{hist}(X))$.

If X contains duplicates, recall that $\mathrm{reg}(X) \leq \mathrm{reg}(\overline{X})$ and $\mathrm{hist}(X) = \mathrm{hist}(\overline{X})$. Applying the previous argument to the permutation $\overline{X}$ proves the claim. ◀

▶ **Lemma 10.** $\mathrm{reg} \preceq \mathrm{HSMS}$.

Proof. Let $X = X_1 \sqcup \cdots \sqcup X_k$ be any partition into monotone subsequences. For each $X_i = (X[a_1], \ldots, X[a_t])$, where $a_1 < \cdots < a_t$, set $f(a_j) = a_{j-1}$ for $j > 1$, and $f(a_1) = 1$ if $a_1 > 1$. This defines a finger function. Recall the edge weights $w(\{i,j\}) = \log(2+|i-j|+|X[i]-X[j]|)$ from Theorem 5. Since X_i is monotone, and all values are ranks in $[n]$, we have

$$\sum_{j=2}^{t} \left((a_j - a_{j-1}) + |X[a_j] - X[a_{j-1}]|\right) \leq 2n. \text{ Thus, by concavity of the logarithm, for } t > 1,$$

$$\sum_{j=2}^{t} w(\{a_j, a_{j-1}\}) \leq (t-1)\log\left(2 + \frac{2n}{t-1}\right) \in O\left(t\left(1 + \log\frac{n}{t}\right)\right).$$

The possible additional edge $\{a_1, 1\}$ has weight $O(\log n)$. Since $\log n \leq t(1 + \log(n/t))$, the total weight contributed by X_i is $O(t(1 + \log(n/t)))$, also when $t = 1$. Summing over all subsequences gives $n + \log \operatorname{reg}(X) \in O\left(n + \sum_{i=1}^{k} |X_i| \log \frac{n}{|X_i|}\right) = O\left(n(1 + H(X_1, \ldots, X_k))\right)$. Taking the minimum over all monotone partitions shows $\operatorname{reg} \preceq \mathrm{HSMS}$. ◀

Finally, we show in Appendix C.2:

▶ **Theorem 11.** *If X is a permutation, then* $\log \operatorname{reg} \in O(n + \log \mathrm{Reg})$.

3 SproutSort in the BST model

We study the BST model where the input X is a permutation of $[n]$ with n a power of two. Note that we can transform any input X' with distinct keys into X in Reg-adaptive time by sorting X' using [3] and padding. We show a BST-access algorithm that runs in reg-adaptive time. We extend the algorithm to allow X to have duplicate keys in Section 3.3.

The offline BST model. We study the cost of accessing X in the offline BST model [15,35]. This model maintains a *single* BST containing exactly the distinct values occurring in X. Each value is stored at one node; at every node, all values in the left subtree are smaller and all values in the right subtree are larger. The algorithm knows X in advance and must access $X[1], X[2], \ldots, X[n]$ in order. Every access starts with a pointer at the root and must visit the node storing the requested value. At unit cost, the algorithm may move the pointer to its parent or either child, or rotate the current node with its parent. The tree persists between accesses. The total cost is n plus the number of pointer moves and rotations.

BST merge sort. A *BST merge sort* [8] operates on *multiple* BSTs and builds a BST that contains the input values. A size-one input yields a one-node BST. Otherwise, the algorithm picks $k \in [n-1]$, recursively computes BSTs T_1 and T_2 for $X[1..k]$ and $X[(k+1)..n]$, and combines them by a *BST merge*: this operation intuitively merges two trees T_1 and T_2 by selecting a connected set of nodes S_1 and S_2 (that each contain the root) and arbitrarily re-arranging their parent and child pointers such that the result is a BST that contains the elements of T_1 and T_2. The cost of this merge is then $|S_1| + |S_2|$. Formally, for T_1 and T_2, mark a connected set of nodes S_1 and S_2 containing the root and the *block boundaries*: nodes in T_1 and T_2 whose predecessor or successor in X lies in the other T_i. The merge returns a BST T on exactly the nodes of T_1 and T_2, minus duplicates, such that $S := S_1 \cup S_2$ is connected and contains the root of T. The total cost is the sum of all merge costs. Blelloch and Dobson [8] show a strong link between merge sort and the offline BST model:

▶ **Theorem 12** (Theorem 6 in [8]). *A BST merge sort that sorts a permutation X with cost $f(X)$ induces an offline BST algorithm that accesses $X[1], X[2], \ldots, X[n]$ with cost $O(f(X))$.*

Their same proof directly applies to input lists with duplicates! Thus, to obtain good access bounds in the BST model, we may focus on designing a BST merge sort algorithm.

3.1 SproutSort as an algorithm

We first introduce SproutSort as a classical algorithm. In Section 3.2 we change it into a BST merge sort. Classical merge sort (Algorithm 1) recursively sorts the two halves of X and merges them in linear time. On X, the merge-sort recursion tree R is then a complete binary tree with n leaves. Our core idea is to store the sorted sublists corresponding to recursive calls as a forest of *sprout trees*, allowing some merges to take sublinear time.

Algorithm 1 Recursive Merge Sort

function MERGESORT(X)
  if $|X| = 1$ **then return** X
  $n \leftarrow |X|$
  $X_\ell \leftarrow$ MERGESORT($X[1], \ldots, X[\frac{n}{2}]$)
  $X_r \leftarrow$ MERGESORT($X[\frac{n}{2}+1], \ldots, X[n]$)
  return Merge(X_ℓ, X_r)

Sprout forests. Let T be a complete binary tree with leaves $1, \ldots, n$, and associate the element $X[i]$ with the leaf $X[i]$. The *height* of a node $u \in T$ is the path length from u to a leaf in its subtree. For an integer $k \geq 0$ and a subset $Y \subseteq X$, the *sprout forest* $T_k[Y]$ is the subgraph of T induced by all nodes $u \in T$ on the root-to-leaf path of some leaf in Y that have height at most k. In particular, the set of leaves of $T_k[Y]$ is precisely Y. See Figure 2. To represent $T_k[Y]$, for a fixed pair (k, Y), store a root list with all roots of $T_k[Y]$ in-order. Store each tree in $T_k[Y]$ as a pointer structure where vertices in $T_k[Y]$ are nodes with two pointers to their left and right children. Each node $u \in T_k[Y]$ stores the index of the smallest and largest leaf in its subtree, plus a pointer to the node corresponding to u in T.

Our sprout trees support two operations. GROW($T_k[Y]$) for $k \geq 0$ and $Y \subseteq X$ computes $T_{k+1}[Y]$. MERGE($T_k[Y_1], T_k[Y_2]$) for $k \geq 0$ and $Y_1, Y_2 \subseteq X$ computes $T_k[Y_1 \cup Y_2]$ (see Figure 3). Both operations consume their input forests. We analyse sequences of these operations with the accounting method [14]. We assign one coin to each node which can pay for $O(1)$ work. The amortized running time of MERGE is $O(m)$ where m is the number of trees in the output.

ER: suggest to phrase it as "holds when the input has no duplicates"

▶ **Theorem 13.** *Consider any sequence of* GROW *and* MERGE *operations, with* $k \leq \log n$*, starting with a collection of sprout forests that have* n *nodes in total. Let* m_i *be the size of the root-list of the forest produced by operation* i*. The total running time is* $O\left(n + \sum_i m_i\right)$*. Moreover,* MERGE *never increases the number of nodes.*

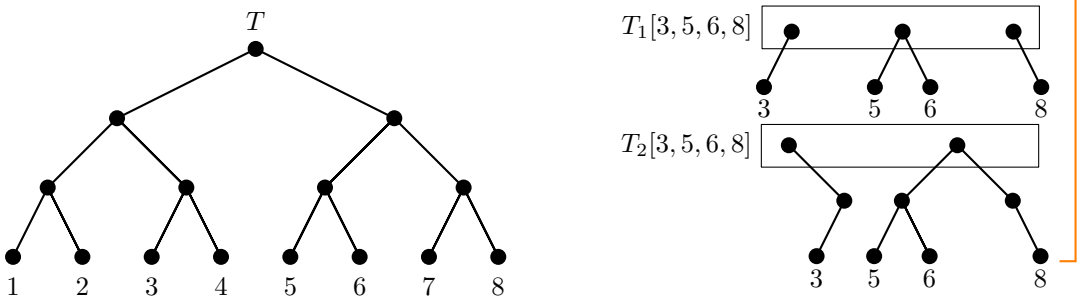


Figure 2 A complete binary tree on X and a sprout forest before and after GROW.

Algorithm 2 Recursively merging sprout trees.

function TreeMerge(T_1, T_2)
  if T_1 is nil **then return** T_2
  if T_2 is nil **then return** T_1
  return a new node with children TreeMerge($T_1.\ell, T_2.\ell$) and TreeMerge($T_1.r, T_2.r$)

Proof. Initially, we create one coin for each existing node. Whenever we remove a node, we spend its coin to do $O(1)$ work. Additionally, the i-th operation can take $O(m_i)$ time.

A Grow($T_k(Y)$) operation scans all roots in $T_k(Y)$. For each root r it encounters, it either creates a new root with r as its singular child or a new root with two children, one of which is r. Thus, Grow inspects at most $3m_i$ nodes, and, since the new root list follows the in-order traversal of the old root list, Grow takes $O(m_i)$ time. Grow also creates m_i coins.

For Merge($T_k(Y_1), T_k(Y_2)$), observe that each root in $T_k(Y)$ is at least a root in either $T_k(Y_1)$ or $T_k(Y_2)$, and at most a root in both. Thus, the total number of trees in $T_k(Y_1)$ and $T_k(Y_2)$ is at most $2m_i$, and since root-lists of $T_k(Y_1)$ and $T_k(Y_2)$ are in-order, we can identify these roots in $O(m_i)$ time. Having identified colliding roots, we invoke our TreeMerge procedure on all trees that share a root (see Algorithm 2). Whenever this recursive merge replaces two nodes by one, we transfer one node's coins to the replacement and spend the other's on the $O(1)$ work needed for this merge. TreeMerge does not create any coins.

In total, we create $O(n+\sum_i m_i)$ coins and spend $O(m_i)$ time beyond the coin budget. ◀

SproutSort We now design an algorithm SproutSort that uses Grow and Merge to compute $T_k[X]$ where $|X| = 2^k$. Similar to recursive merge sort, the algorithm splits $X = X_\ell \sqcup X_r$ into equal halves and recursively computes $T_{k-1}[X_\ell]$ and $T_{k-1}[X_r]$. It then returns $T_k[X] = \text{Grow}(\text{Merge}(T_{k-1}[X_\ell], T_{k-1}[X_r]))$. See Algorithm 3.

▶ **Definition 14.** *Recall that a* finger function *is a map $f : [n] \setminus \{1\} \to [n]$ satisfying $f(i) < i$. For $i \geq 2$, we define the graph distances:*

- $\texttt{dist}_R(i, f(i))$*, between leaves i and $f(i)$ in the complete binary tree R on $[n]$.*
- $\texttt{dist}_T(i, f(i))$*, between leaves $X[i]$ and $X[f(i)]$ in T.*

For example, for $n \geq 4$, $\texttt{dist}_R(\frac{n}{2}, \frac{n}{2}+1) = 2\log n$ and $\texttt{dist}_R(\frac{n}{2}+1, \frac{n}{2}+2) = 2$.

▶ **Lemma 15.** *Let $n \geq 2$ and let f be any finger function. The running time of SproutSort is $O\big(\sum_{i=2}^{n}(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$.*

Proof. Let g be the total number of nodes created by Grow. Observe that Grow creates one node per tree in its output forest, Merge does not increase the number of nodes, and both Grow and Merge reduce the size of the root list by at most a factor 2. Hence, an invocation of Grow(Merge(...)) in SproutSort that creates g_i nodes, involves root lists of size at most $4g_i$. Thus, by Theorem 13, the total running time is $O(n + \sum_i g_i) = O(n + g)$.

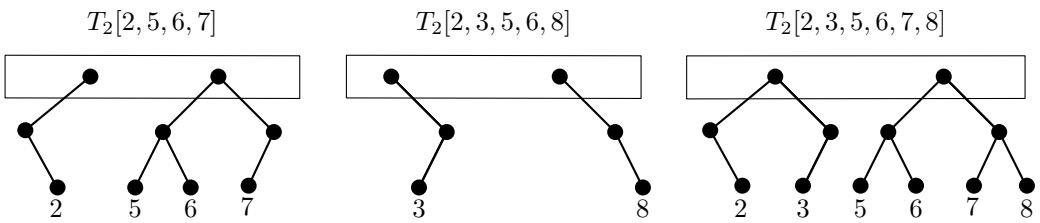


Figure 3 Two sprout forests and their merge.

Whenever Grow creates a node u, we denote by subtree(u) the resulting sprout tree rooted at u. We charge the creation of u to the smallest $i \geq 2$ such that $X[i] \in$ subtree(u) but $X[f(i)] \notin$ subtree(u). If no such i exists, we charge u to item 1. This happens only if $X[1] \in$ subtree(u), which itself happens for at most one node u per level of recursive depth in our algorithm, for a total charge of $O(\log n)$ to item 1.

For all other $i \geq 2$, we bound the charge they receive. At every recursion height $k \geq \texttt{dist}_R(i, f(i))/2$, items i and $f(i)$ belong to the same recursive input X'. Thus, they belong to the same sprout forest $T_k(X')$. If, furthermore $k \geq \texttt{dist}_T(i, f(i))/2$ then they also lie in the same sprout tree of $T_k(X')$. Item i participates in only one Grow operation per recursive height, and stops being charged once items i and $f(i)$ share a sprout tree. Item i therefore receives at most $O(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))$ charges. Consequently, $g \in O\big(\log n + \sum_{i=2}^{n}(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$. The sum is at least $n - 1$, so it absorbs the $O(n)$ initialization cost and the $O(\log n)$ charges to item 1. ◀

▶ **Lemma 16.** *Let X be a permutation of $[n]$ and let f be any finger function. Then $\sum_{i=2}^{n} \texttt{dist}_R(i, f(i)) \in O(n + \sum_{i=2}^{n} \log(i - f(i)))$ and $\sum_{i=2}^{n} \texttt{dist}_T(i, f(i)) \in O(n + \sum_{i=2}^{n} \log |X[i] - X[f(i)]|)$.*

Proof. We prove the bound for $\texttt{dist}_T$, which is the more difficult case because $X[f(i)]$ may lie on either side of $X[i]$. Construct T from the leaves upwards. In round k, the leaves are partitioned into groups of 2^k consecutive values. If two leaves start belonging to the same group in round t, their lowest common ancestor is t edges above them. Thus, $\texttt{dist}_T(i, f(i))$ is twice the number of rounds in which $X[i]$ and $X[f(i)]$ belong to different groups.

Write $\delta_i = |X[i] - X[f(i)]|$. For a fixed i, there are at most $1 + 2\log \delta_i$ rounds with $2^k \leq \delta_i^2$. We bound the remaining rounds by counting how often they occur across all indices.

Fix a round k with $2^k > \delta_i^2$, and suppose $X[i]$ and $X[f(i)]$ belong to different groups. Let $[a, b]$ be the group containing $X[i]$. If $X[f(i)] < a$, then $X[i] - a + 1 \leq \delta_i$; if $X[f(i)] > b$, then $b - X[i] + 1 \leq \delta_i$. Hence $X[i]$ is among the first or last $\delta_i < 2^{k/2}$ values of its group. There are $n/2^k$ groups, and X is a permutation, so this happens for at most $2n/2^{k/2}$ indices in round k. Summing over all rounds gives at most $2n \sum_{k=0}^{h-1} 2^{-k/2} = O(n)$ such occurrences.

The first kind of round contributes $O\big(n + \sum_{i=2}^{n} \log \delta_i\big)$ occurrences, and the second contributes $O(n)$. Each occurrence contributes two to the distance, proving the lemma. ◀

▶ **Theorem 17.** *If X is a permutation and T is a complete binary tree, then the running time of SproutSort is $O(n + \log \text{reg})$.*

Proof. Let f be an arbitrary finger function. By Lemma 15, the running time is $O\big(\sum_{i=2}^{n}(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$. By Lemma 16, the running time is therefore upper-bounded by $O\big(n + \sum_{i=2}^{n} \log(i - f(i)) + \sum_{i=2}^{n} \log |X[i] - X[f(i)]|\big)$. By the definition of reg, there exists some finger function f such that $O(n + \log \text{reg}) = O\big(n + \sum_{i=2}^{n} \log(i - f(i)) + \sum_{i=2}^{n} \log |X[i] - X[f(i)]|\big)$ and so the theorem follows. ◀

3.2 Running SproutSort in the BST Model

What remains is to implement SproutSort as a BST merge sort algorithm, which then implies an algorithm for BST access. That is, we must implement our recursive algorithm so that the output of each recursive call is a BST whose nodes are exactly the input of the call. The input is a list of singleton BSTs whose left-to-right order is X. Each recursive call combines its two output BSTs using the Merge operator from the beginning of the section.

Consider a recursive call of SproutSort at recursion height k on a contiguous sublist X' of X. To represent a sprout tree $T' \in T_k[X']$ as a BST, first contract all its maximal unary

Algorithm 3 Recursive SproutSort.

function SproutSort(X)
  if $|X| = 1$ **then return** $T_0[X]$
  $m \leftarrow |X|,\ k \leftarrow \log_2 m$
  $X_\ell \leftarrow X[1], \ldots, X[m/2],\ X_r \leftarrow X[m/2+1], \ldots, X[m]$
  $T_{k-1}[X_\ell] \leftarrow$ SproutSort(X_ℓ)
  $T_{k-1}[X_r] \leftarrow$ SproutSort(X_r)
  return Grow(Merge($T_{k-1}[X_\ell], T_{k-1}[X_r]$))

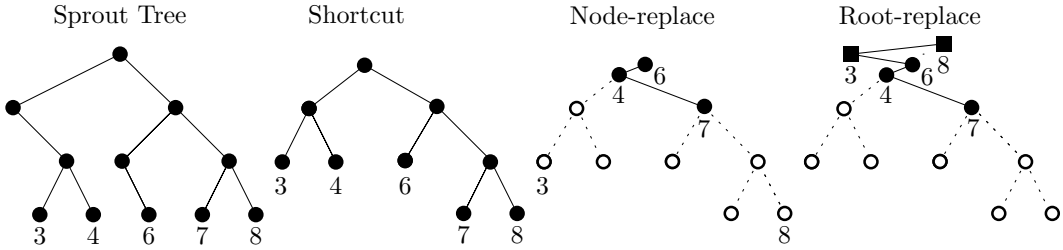


Figure 4 Converting a sprout tree to a BST in three steps. White nodes are implicit; each root-to-leaf path has a corresponding BST traversal.

paths, yielding a tree T''. Assign each internal node u of T'' the key of the rightmost leaf in its left subtree, unless this subtree consists of a single leaf. Similarly, assign it the key of the leftmost leaf in its right subtree, unless this subtree consists of a single leaf. Split every node that is assigned two keys, putting the larger key above the smaller. Remove the original leaves and any internal nodes that were not assigned a key, yielding a BST T''' containing all keys except the leftmost and rightmost. Finally, make the rightmost key v a new root, the leftmost key u the left child of v, and T''' the right subtree of u. A one-leaf sprout tree becomes a singleton BST; see Figure 4. To represent a forest $T_k[X']$, simply represent each sprout tree in this way and connect the roots in a linked list via their right child pointers. This yields a BST, as the root list of a sprout forest contains the items in increasing order.

Our operations. The Grow operation traverses the root list of a sprout forest and may attach pairs of adjacent trees to a new root. In the BST representation, this corresponds to traversing the root list through right-child pointers and performing $O(1)$ rotations per pair. The number pointer traversals and rotations performed is bounded by the root list size.

The Merge operation, when given two disjoint BSTs, traverses their root lists and calls TreeMerge (Algorithm 2) on sprout trees that share the same root. Each recursive call to TreeMerge separates the root of each sprout subtree T_i from its children $T_i.\ell, T_i.r$, recursively merges the left and right children, and then re-attaches the results to a new root. In the BST representation, separating its root from its two children involves up to four nodes near the root. Re-attaching to a new root is the reverse (see Figure 5). For each separation operation, we mark the roots of the resulting binary search trees, plus their left children (nodes u, v, x, y in Figure 5). At the start of Merge, we also mark all nodes on the right spine of each input BST (i.e., the root list plus their left children). With these markings, the marked nodes of each input BST form a connected subtree containing its root, and unmarked subtrees remain unchanged. Moreover, every block boundary between the two input BSTs is exposed during TreeMerge and therefore marked. Thus, Grow(Merge(...)) in SproutSort is a BST merge. The number of marked nodes is bounded by the number of

root-list entries traversed and nodes processed by TreeMerge. By Theorem 13 and our analysis of SproutSort, the total BST merge cost is $O(n + \log \mathrm{reg}(X))$, which shows:

▶ **Theorem 18.** *SproutSort is a BST merge sort [8]. Thus, it yields an offline BST algorithm for accessing a permutation X in $O(n + \log \mathrm{reg}(X))$ node traversals and rotations.*

3.3 Duplicate Items

So far, we assumed that X has no duplicates. We now show that SproutSort can be adapted to the case where X contains duplicate items. In the offline BST model for accessing elements, this corresponds to having an access sequence X that contains the same access multiple times, where the BST stores the set Y of distinct keys in X. Although input elements can only be compared, we can transform any input X' into a list $X \in [U]^n$, where U is the number of distinct keys in X (via [3]) in $O(n + \log \mathrm{reg})$ time. Furthermore, we can assume that we know for each $x \in [U]$ how many times x occurs in X.

▶ **Definition 19.** *We denote by T a specific binary tree whose leaves are $1, \ldots, U$ in-order and have the same depth. T could have many shapes, as we allow nodes with only one child. We construct T cleverly later, and associate each $X[i]$ with the leaf that stores this value.*

Given a specifically-shaped, currently-unspecified T, we run SproutSort as before where we use T as the skeleton for the **Sprout Trees**, as opposed to the complete binary tree. Each recursive call on $X[i..j]$ returns a sprout forest of the distinct values in $X[i..j]$. Separate recursive calls store separate copies of their forests, and different forests may contain leaves with the same value. Each leaf with value $x \in [U]$ stores a linked list of the occurrences of x in the corresponding recursive input $X[i..j]$. When TreeMerge encounters two copies of the same leaf, it replaces them by one and concatenates their occurrence lists in $O(1)$ time.

Recreating Theorem 13 and Lemma 15 The amortisation proof of Theorem 13 applies as-is: identifying two copies of a leaf spends the deleted copy's coin on the $O(1)$ work required to identify the duplicate, and the linked list merge. We do have to extend Lemma 15:

▶ **Lemma 20.** *Let T be the tree from Definition 19, f be any finger function and $n \geq 2$. SproutSort on input $X \in [U]^n$ runs in time $O\big(n + \sum_{i=2}^{n} (\log(i - f(i)) + \mathtt{dist}_T(i, f(i)))\big)$.*

Proof. Let g be the total number of nodes created by Grow. Let g_j be the number of nodes created by Grow in the j-th recursive call; then $g := \sum_j g_j$. If Grow creates nodes, it creates one per tree in its output forest. Otherwise, its input forest already contains the root of T, and it returns this single tree unchanged. Thus, its output root list has size at most $g_j + 1$. Since Grow combines at most two input trees into each output tree, the preceding Merge has an output root list of size at most $2(g_j + 1)$. By Theorem 13, the total running time is $O(n + \sum_j (g_j + 1)) = O(n + g)$, since there are $n - 1$ non-base recursive calls.

Whenever Grow creates a node u in a recursive call on $X[a..b]$, let $C_u := \{i \in [a, b] \mid X[i] \in \mathrm{subtree}(u)\}$ be all input keys in the current recursive call that lie below u. We charge the creation of u to the smallest $i \geq 2$ such that $i \in C_u$ but $f(i) \notin C_u$. If no such i exists, the smallest index in C_u must be 1, since $f(i) < i$. We then charge u to item 1. At each recursion height, only one recursive input contains item 1, and only one newly created sprout tree contains its leaf. Thus, item 1 receives $O(\log n)$ charges.

For all other $i \geq 2$, at every recursion height $k \geq \mathtt{dist}_R(i, f(i))/2$, items i and $f(i)$ belong to the same recursive input. If, furthermore, $k \geq \mathtt{dist}_T(i, f(i))/2$, their leaves belong to the same sprout tree. Item i therefore receives at most $O(1 + \mathtt{dist}_R(i, f(i)) + \mathtt{dist}_T(i, f(i)))$

charges. This also holds when $X[i] = X[f(i)]$, in which case $\texttt{dist}_T(i, f(i)) = 0$. Every node created by GROW charges exactly one item, so the total number of charges is $\sum_j g_j$. Summing the bounds above, including the $O(\log n)$ charges to item 1, gives $\sum_j g_j \in O\big(\log n + \sum_{i=2}^{n}(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$. Since we could upper bound our running time by $O(n + \sum_j g_j)$, we can now upper bound our running time by $O\big(n + \sum_{i=2}^{n}(\texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$. What remains is to replace all $\texttt{dist}_R$ terms. For this, we can directly invoke Lemma 16, as R is completely unaffected by X containing duplicates. ◀

Choosing the tree of Definition 19. By Lemma 20, what remains is to bound the total distance between leaves representing $X[i]$ and $X[f(i)]$ in our custom-made tree T. We choose T using the number of occurrences of each value via some weight function:

▶ **Lemma 21.** *Let $w : [U] \to [n]$ be a weight function with $\sum_{x=1}^{U} w(x) = n$. There is a binary tree T, possibly containing nodes with one child, whose leaves are $1, \ldots, U$ in-order and have the same depth, such that, putting $E(x) := \max_{y \in [U]}\{\texttt{dist}_T(x, y) - 8 \log \max\{1, |x - y|\}\}$, we have $\sum_{x=1}^{U} w(x)E(x) \in O(n)$. T has $O(U)$ nodes and can be constructed in $O(U)$ time.*

Proof. For an integer interval $[a, b] \subseteq [U]$, we construct a tree $T'_{a,b}$ whose leaves are $a, \ldots, b$. If $a = b$, then $T'_{a,b}$ consists of a single leaf labelled a. Otherwise, choose an index $r \in \{a, \ldots, b-1\}$ by the rule below. Construct $T'_{a,r}$ and $T'_{r+1,b}$ recursively, and make their roots the left and right children of a new root. The resulting tree is $T'_{a,b}$. Finally, put $T' = T'_{1,U}$.

We choose r as follows. Put $J = [a, b]$, $m = b - a + 1$, and $W = \sum_{x \in J} w(x)$. Require $m/3 \le r - a + 1 \le 2m/3$, so both child intervals contain at most $2m/3$ keys. Put $q = \lceil \sqrt{m} \rceil$. Starting with the smallest permitted index r_0, consider $r_0, r_0 + 2q, r_0 + 4q, \ldots$, stopping when the index is no longer permitted. For each candidate r, put $B_r = J \cap [r - q + 1, r + q]$. These are disjoint, and there are $\Theta(\sqrt{m})$ candidates with total weight at most W, so one has weight $O(W/\sqrt{m})$. Choose an r minimizing this weight, and call the keys in B_r *bad for* $T'_{a,b}$.

We now turn T' into T by lengthening its edges. For each node u of T', let m_u be its number of descendant leaves and assign it the target height $h(u) = \lfloor 2 \log m_u \rfloor$. Replace every parent–child edge pu by a path of $h(p) - h(u)$ edges, inserting nodes with one child. This length is positive because $m_u \le 2m_p/3$. Along any path from an original node u to a descendant leaf, the new edge lengths telescope to $h(u)$, since leaves have target height zero. Thus all leaves have the same depth. What remains is to bound $\sum_x w(x)E(x)$ in T.

For distinct leaves x, y, their lowest common ancestor is the same node u in both trees, and $\texttt{dist}_T(x, y) = 2h(u)$. Suppose leaves in the subtree of u form an interval of size m, partitioned at the chosen index r. The keys x, y lie in different child intervals. If $x \notin B_r$, then $|x - y| \ge q \ge \sqrt{m}$, so $\texttt{dist}_T(x, y) = 2h(u) \le 4 \log m \le 8 \log |x - y|$. This pair contributes no positive excess to $E(x)$. If $x \in B_r$, its excess is at most $4 \log m$. Since $y = x$ contributes zero, $E(x)$ is at most the sum of $4 \log m$, over all nodes at which x is bad.

Consider a node whose descendant interval has size m and weight W. Its bad keys have weight $O(W/\sqrt{m})$. The contribution of these bad keys to the weighted sum is therefore $O(W \log m/\sqrt{m})$. Charge each key x (good or bad) in this interval $O(w(x) \log m/\sqrt{m})$. The nodes whose intervals contain x are the ancestors of x in T', and their interval sizes shrink by a factor at most $2/3$. For each k, there are $O(1)$ nodes with $2^k \le m < 2^{k+1}$. Hence the total charge to x is $O(w(x) \sum_{k \ge 1}(k + 1)2^{-k/2}) = O(w(x))$. Summing over x gives $O(n)$.

To get the running time, compute for $t \in [U]$ the prefix sums $P(t)$ where $P(0) = 0$ and $P(t) = \sum_{x=1}^{t} w(x)$ in $O(U)$ time. The weight of each candidate's bad interval $[L, R]$ is then $P(R) - P(L - 1)$, which takes $O(1)$ time to compute. Each leaf $T'_{a,a}$ is created in constant time. At an internal node with m descendant leaves, choosing r takes $O(\sqrt{m})$ time, and

lengthening its outgoing edges inserts $O(\log m)$ nodes. There are $O(U/2^k)$ nodes of T' with $2^k \leq m < 2^{k+1}$: each has at least 2^k descendant leaves, and each leaf lies below only $O(1)$ such nodes. Summing over k gives $O(U)$ construction time and nodes. ◀

▶ **Theorem 22.** *Let $X \in [U]^n$, where we know for each element $x \in X$ its multiplicity in X. With T constructed by Lemma 21, SproutSort sorts X in $O(n + \log \mathrm{reg}(X))$ time.*

Proof. For each $x \in [U]$, let $w(x)$ be its multiplicity in X, and construct T by Lemma 21; this takes $O(n)$ time. Let f be any finger function and put $\delta_i := \max\{1, |X[i] - X[f(i)]|\}$. Since all leaves of T have the same depth, $\mathtt{dist}_T(i, f(i)) = 2\,\mathrm{height}(\mathrm{LCA}_T(X[i], X[f(i)])) \leq 8 \log \delta_i + 2E(X[i])$. This includes equal-value pairs, whose tree distance is zero. Furthermore, $\sum_{i=2}^{n} E(X[i]) \leq \sum_{x=1}^{U} w(x)E(x) \in O(n)$. By Lemma 20, the running time is therefore $O\big(n + \sum_{i=2}^{n} \log(i - f(i)) + \sum_{i=2}^{n} \log \delta_i\big)$. By the definition of reg, there is a finger function f for which these two sums equal $\log \mathrm{reg}(X)$. Traversing the output leaves and their occurrence lists takes $O(n)$ additional time and yields a sorted ordering of X. ◀

Finally, we transform the sorting algorithm into a BST access algorithm in the same manner as in Section 3.2. We use the BST representation and markings of Section 3.2, with one node per distinct key in each recursive output. When TreeMerge identifies two copies of a leaf, both corresponding BST nodes are marked before they are identified. Thus, every key present in both input trees is marked in both. Each input BST contains one node per distinct key. However, the same key may occur in both input trees, so a merge may encounter two nodes storing that key. We mark both nodes and combine them into a single node in the output BST. For example, if both input trees contain a node storing 3, the output tree contains only one node storing 3. Both input nodes count towards the merge cost. Unmarked subtrees retain their nodes and children. Combining equal-key nodes does not remove occurrences from the access sequence: the resulting BST still accesses a key once for each occurrence in X.

The extension preserves the access bound of Theorem 12. In Blelloch and Dobson's construction [8], marked nodes with the same key produce identical geometric points. Unmarked subtrees contain no shared keys, so their structure and relative priorities remain unchanged. The resulting set is therefore still arborally satisfied, with $O(n)$ points plus $O(1)$ per marked input node, giving the same asymptotic BST access cost.

▶ **Theorem 23.** *SproutSort yields a deterministic offline BST algorithm for accessing X with cost $O(n + \log \mathrm{reg}(X))$, also when X contains duplicates.*

Proof. We use the BST representation and markings of Section 3.2, identifying marked copies of the same key as described above. Each recursive call performs a BST merge with the duplicate extension described above. The number of marked input nodes is bounded by the work performed by Grow and Merge. By Theorem 22, the total merge cost is therefore $O(n + \log \mathrm{reg}(X))$. The duplicate extension of Theorem 12 then gives an offline BST algorithm with the claimed access cost. ◀

A SkeletonSort with no duplicates

Let X be an input list of arbitrary comparable elements that contains no duplicates. We present a simple, randomised algorithm that sorts X in expected $O(n+\log \mathrm{reg})$ time, without any prior knowledge of the ranks of elements in X. Our algorithm is based on Quicksort, which picks a uniformly random pivot p and partitions X according to whether elements are smaller or larger than p (recursing on each part). We design a tree data structure that supports both pivot sampling and partitioning, in expected $O(n + \log \mathrm{reg})$ total time. By Theorem 11, the algorithm is not only reg-adaptive but also Reg-adaptive.

Algorithm 4 Quicksort.

```
function QUICKSORT(X)
    if X is empty then
        return an empty list
    p ← random-item(X)
    L_ℓ ← QUICKSORT({x ∈ X : x < p})
    L_r ← QUICKSORT({x ∈ X : x > p})
    return the concatenation of L_ℓ, the singleton list [p], and L_r
```

Skeleton Trees. We again assume that $n = 2^k$ is a power of two (we may always pad the input with distinct increasing sentinel items declared larger than every key in X, without it asymptotically changing $O(n + \log \mathrm{reg})$). Let R be the complete binary tree whose leaves are the indices $1, \ldots, n$ in-order. For a subset of indices $I \subseteq [n]$, we define the *I-skeleton of R*, denoted R_I, as the tree obtained by keeping the leaves in I and their ancestors, and shortcutting every node with only one child. For $I = \emptyset$, put $R_I = \mathrm{nil}$. We call such trees *skeleton trees*, and they are the basis of our SkeletonSort algorithm. Each internal node of a skeleton tree has exactly two children, and each node u corresponds to an original node of R; the *raw height* of u is the number of edges from that original node to any of its descendant leaves in R. We store each skeleton tree as a pointer structure with child and parent pointers. A leaf labelled i stores a pointer to the input item $X[i]$. Each node stores its number of descendant leaves, together with the minimum and maximum of the input values $X[i]$ represented by those leaves. For each internal node, these fields can be computed from the two children in constant time.

Random sampling. To simulate Quicksort, we need for any $I \subset [n]$ to be able to generate a uniformly random index $i \in I$. We show how to do this given R_I: the root of this tree has stored $|I|$. Choose a uniformly random integer $j \in [|I|]$ and find the j-th leaf in-order, using the leaf counts to choose the next child. This procedure Random-Leaf returns a pointer to an item sampled uniformly from I and takes time proportional to the number of nodes on

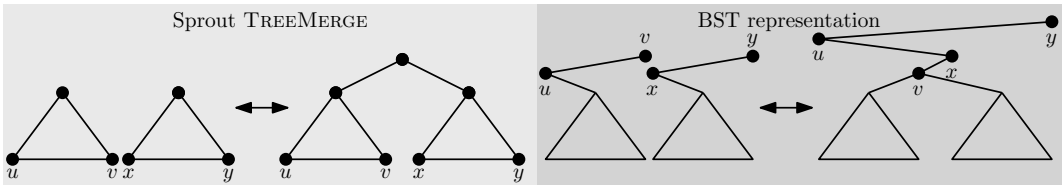


Figure 5 The primitives used in Sprout Tree-Merge (left) and their BST representation (right).

its search path. We allow constant-time arithmetic on $O(\log n)$-bit counters and generation of a uniformly random integer in a specified range of size at most n.

Splitting along a pivot. Given some random index $i \in I$, this yields some pivot value $p = X[i]$. Quicksort then splits the input along this pivot. We simulate this split by defining $I^{<p} = \{i \in I : X[i] < p\}$ and $I^{=p}$ and $I^{>p}$ analogously. The operation Skeleton-Split(R_I, p) (Algorithm 5) consumes its input and returns the three disjoint skeletons $R_{I^{<p}}$, $R_{I^{=p}}$, and $R_{I^{>p}}$. In this code, whenever Create-Node creates a node, it updates the stored fields and the parent pointers of its children in constant time. Every returned non-nil tree has its root's parent pointer reset to nil. When both children are non-nil, the new node corresponds to the same original node of R as the node currently being partitioned.

Algorithm 5 Partitioning skeleton trees.

function Create-Node(S_ℓ, S_r)
  if both S_ℓ and S_r are nil **then**
    return nil
  if exactly one of S_ℓ, S_r is nil **then**
    return the non-nil tree
  $u \leftarrow$ a new node with children S_ℓ and S_r
  Update the stored fields of u and the parent pointers of its children
  return u

function Skeleton-Split(S, p)
  if S is nil **then**
    return nil, nil, nil
  if all leaves i of S satisfy $X[i] = p$ **then**
    return nil, S, nil
  if all leaves i of S satisfy $X[i] < p$ **then**
    return S, nil, nil
  if all leaves i of S satisfy $X[i] > p$ **then**
    return nil, nil, S
  $S_\ell^<, S_\ell^=, S_\ell^> \leftarrow$ Skeleton-Split$(S.\ell, p)$
  $S_r^<, S_r^=, S_r^> \leftarrow$ Skeleton-Split$(S.r, p)$
  $S^< \leftarrow$ Create-Node$(S_\ell^<, S_r^<)$
  $S^= \leftarrow$ Create-Node$(S_\ell^=, S_r^=)$
  $S^> \leftarrow$ Create-Node$(S_\ell^>, S_r^>)$
  Discard the old node S
  return $S^<, S^=, S^>$

A.1 SkeletonSort.

Given the above two primitives, we can simulate *Quicksort* without using linear time per recursive call: We construct $R_{[n]} = R$ and its stored fields in $O(n)$ time, then run SkeletonSort on its root (Algorithm 6). Since the values are distinct, $S^=$ consists of the single leaf storing the pivot. Output lists have head and tail pointers, so each concatenation takes $O(1)$ time.

To analyse this algorithm, fix a random seed and form a tree T_0 as follows. Create one node for every non-empty recursive invocation of SkeletonSort. Its children are the nodes

for its non-empty recursive invocations, and the node is labeled by its pivot. Then, T_0 is a binary search tree that contains the values of X in sorted order.

For each node v of T_0, let $h(v)$ be the height of the subtree of v, i.e. the maximum number of edges from v to a descendant leaf. The node v is labeled by some pivot x. Remove this label and instead attach a path of length $h(v)$ to v whose leaf is labeled by x. Finally, replace every parent–child edge vu by a path of $h(v) - h(u)$ edges, inserting nodes with one child, and call the resulting tree T. Every path from an original node v to a descendant leaf now has length $h(v)$, so all leaves of T have the same depth. The resulting tree T is used only in the analysis; the algorithm does not construct it.

We again use graph distances and slightly overload our notation from previous sections. Here, $\texttt{dist}_R(i,j)$ is the distance between the index leaves i, j in R which is the complete binary tree, and $\texttt{dist}_T(i,j)$ is the distance between the value leaves $X[i], X[j]$ in T. Recall that a finger function is a map $f : [n] \setminus \{1\} \to [n]$ with $f(i) < i$.

▶ **Lemma 24.** *Fix a random seed and let R and T be as above. For any finger function f, the running time of* SkeletonSort *is $O\big(n + \sum_{i=2}^{n}(\texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))\big)$.*

Proof. Consider a non-empty invocation of SkeletonSort on a skeleton tree S, where Random-Leaf selects leaf i and pivot $p = X[i]$. The subsequent Skeleton-Split must traverse the entire path from the root of S to i, since every subtree along this path contains p. Thus, the time spent on Random-Leaf is bounded by the time spent on Skeleton-Split.

What remains is to bound that time. A node *gets split* if none of the three early stopping conditions apply (in those cases, the call takes constant time and does not recurse). If s nodes get split, the partitioning procedure visits $2s + 1$ nodes and takes $O(1 + s)$ time.

Singleton invocations take $O(1)$ time each, so we consider only invocations with at least two items. Recall that, for our fixed seed, the recursion tree T_0 represents the execution of Quicksort that SkeletonSort simulates. Let r be the node of T_0 corresponding to the current invocation SkeletonSort(S), and let $h(r)$ denote its height in T_0. Color a node of S red if its raw height is at least $h(r)$, and blue otherwise. Since $h(r) \geq 1$, all leaves are blue. If present, the red nodes form a connected subtree containing the root of S, with maximal blue subtrees hanging below it. If there are no red nodes, the entire tree S is the sole maximal blue subtree. If there are b maximal blue subtrees, there are $b - 1$ red nodes, as every red node has two children.

The following analysis is very similar to running SproutSort, and specifically, Theorem 13, in reverse. It might be useful to think of the red nodes as the root list of a sprout forest, and each maximal blue subtree as a sprout tree. (This analogy only works for the analysis. We cannot run SkeletonSort with sprout trees as, due to the random pivot

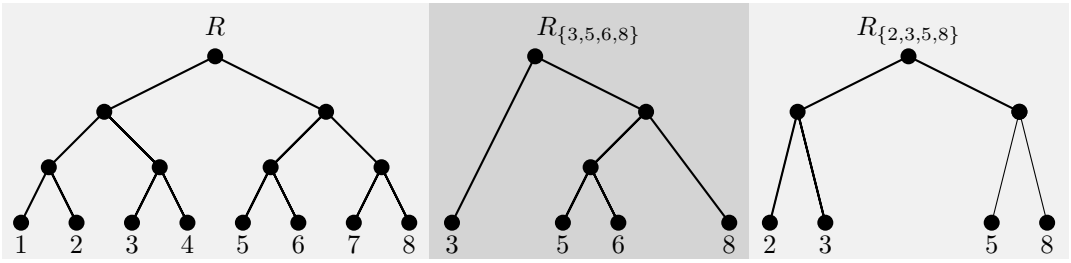


Figure 6 A complete binary tree and two skeleton trees.

selection, we do not know the height $h(r)$ of the current invocation during the algorithm.)

We charge the work of SKELETON-SPLIT to leaf indices of S. Define the *leader* of a subtree of S to be its leftmost leaf. For each of the b maximal blue subtrees, charge its leader once. These b charges cover the constant overhead and the cost of splitting all red nodes. Next, consider a blue node u that gets split by SKELETON-SPLIT. At least two of its three output trees are nonempty. The leader of u remains the leader of one of these output trees. We charge the work at u to the leader of another nonempty output tree. Thus, every blue split is charged to an index that becomes a new leader.

We first consider charges to index 1. It receives at most one red charge per non-empty invocation of SKELETONSORT, and therefore $O(n)$ red charges in total. It receives no blue charges, since whenever its leaf is present, it is already the leader of its subtree.

To bound the remaining charges, fix $i \geq 2$ and consider red charges to i. If the current height $h(r)$ exceeds both $\texttt{dist}_T(i, f(i))$ and $\texttt{dist}_R(i, f(i))/2$, then the current input S contains both indices i and $f(i)$, and their common ancestor in R is blue. They therefore lie in the same maximal blue subtree. Since $f(i) < i$, index i cannot be its leader and receives no red charge. The heights decrease strictly along the recursive path containing i, and i receives at most one red charge per invocation. Hence i receives $O(1+\texttt{dist}_R(i, f(i))+\texttt{dist}_T(i, f(i)))$ red charges.

Next consider a blue split charged to i, and let u be the corresponding original node in R, of raw height k. If the leaf $f(i)$ is not a descendant of u in R, then the lowest common ancestor of i and $f(i)$ is a strict ancestor of u, so $k < \texttt{dist}_R(i, f(i))/2$. Otherwise, consider the output subtree whose new leader is i. Since $f(i) < i$, this subtree cannot contain the leaf $f(i)$. Thus, the current or an earlier Quicksort pivot must have separated $X[i]$ and $X[f(i)]$. Their lowest common ancestor in T_0 is therefore the current invocation or one of its ancestors, so $h(r) \leq \texttt{dist}_T(i, f(i))$. Since the charged node is blue, $k < h(r)$. In either case, $k < \max\{\texttt{dist}_R(i, f(i))/2, \texttt{dist}_T(i, f(i))\}$.

Once a blue split at an original node u is charged to i, the leaf i becomes the leftmost surviving leaf below u in its output tree. Every subsequent invocation containing i operates on a subset of these leaves, so i remains the leftmost surviving leaf below u. In other words, i is only charged when it becomes the leader of a node u, after which i remains the leader of that node as long as u remains blue. Thus, i receives at most one blue charge at each original node $u \in R$. Since there is at most one ancestor of i at each raw height, and all charged nodes have raw height $k < \max\{\texttt{dist}_R(i, f(i))/2, \texttt{dist}_T(i, f(i))\}$, index i receives $O(1 + \texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i)))$ blue charges in total.

Summing the red and blue charges over all indices, and adding the $O(n)$ initialization and output costs, proves the lemma. ◀

▶ **Lemma 25.** *Let X be a list of n distinct elements and let T be as above. For any distinct indices $i, j \in [n]$, we have $\mathbb{E}[\texttt{dist}_T(i, j)] \in O(1 + \log|\operatorname{rank}_X(X[i]) - \operatorname{rank}_X(X[j])|)$.*

Proof. We use the model of random binary search trees [33] where each rank $x \in [n]$ receives an independent uniform priority $p_x \in [0, 1]$, and the root of every subtree is the rank with the largest priority. This has the same distribution as the Quicksort recursion tree T_0.

Let $a < b$ be the ranks of $X[i]$ and $X[j]$ in sorted order, and put $d = b - a$. Note that $d = |\operatorname{rank}_X(X[i]) - \operatorname{rank}_X(X[j])|$. Let k be the rank with the largest priority among $[a, b]$. Then the invocation with pivot k is the lowest common ancestor of $X[i]$ and $X[j]$ in T_0.

Let B be the first rank greater than b whose priority exceeds p_k (or $n + 1$ if no such rank exists). Since k has the largest priority in $[a, b]$, the right subtree of k contains exactly the elements with rank $k + 1, \ldots, B - 1$. Conditioned on k and B, this is a random binary

search tree. The expected height of a random binary search tree with s keys is $\leq 5\log(2+s)$, see [14, Chapter 12.4]. Thus, the expected height of the right subtree, conditioned on k and B, is $\leq 5\log(2+B-a)$. For any $t \in [b,n]$, the event $B > t$ occurs precisely when the largest priority among $[a,t]$ belongs to $[a,b]$. Thus, $\Pr(B > t) = (d+1)/(t-a+1)$. In particular, for every integer $q \geq 0$, $\Pr(B - a > 2^q(d+1)) \leq 2^{-q}$. Summing over these exponentially increasing ranges gives $\mathbb{E}[\log(2+B-a)] \in O(1+\log d)$. Hence the expected height of the right subtree is $O(1+\log d)$. By a symmetric argument, the expected height of the left subtree is also $O(1+\log d)$. The height of k is one larger than the maximum height of the two subtrees, thus $\mathbb{E}[h(k)] \in O(1+\log d)$.

Recall that T is obtained from T_0 by extending edges until all leaves have the same depth. Thus, $\texttt{dist}_T(i,j) = 2h(k)$. Therefore, $\mathbb{E}[\texttt{dist}_T(i,j)] \in O(1+\log d)$, which proves the lemma. ◀

▶ **Lemma 26.** *Let X be a list of n distinct elements and let f be any finger function. Then*

$$\sum_{i=2}^{n} \texttt{dist}_R(i, f(i)) \in O\left(n + \sum_{i=2}^{n} \log(i - f(i))\right),$$

$$\mathbb{E}\left[\sum_{i=2}^{n} \texttt{dist}_T(i, f(i))\right] \in O\left(n + \sum_{i=2}^{n} \log \max\{1, |\operatorname{rank}_X(X[i]) - \operatorname{rank}_X(X[f(i)])|\}\right).$$

Proof. The first bound follows directly from Lemma 16, since R is the complete binary tree whose leaves are the indices $1, \ldots, n$ in-order. For the second bound, we apply linearity of expectation and Lemma 25. ◀

▶ **Theorem 27.** *For any list X of n distinct elements, SkeletonSort sorts X in expected $O(n + \log \operatorname{reg}(X))$ time.*

Proof. Let f be an arbitrary finger function. By Lemma 24, for every fixed random seed, the running time of SkeletonSort is $O(n + \sum_{i=2}^{n}(\texttt{dist}_R(i, f(i)) + \texttt{dist}_T(i, f(i))))$.

Taking expectations and applying Lemma 26, the expected running time is

$$O\left(n + \sum_{i=2}^{n} \log(i - f(i)) + \sum_{i=2}^{n} \log \max\{1, |\operatorname{rank}_X(X[i]) - \operatorname{rank}_X(X[f(i)])|\}\right).$$

By the definition of reg, there exists a finger function f for which these two sums equal $\log \operatorname{reg}(X)$. The theorem follows. ◀

B Notions of presortedness

Table 1 Measures of presortedness and their target adaptive bounds. Throughout, $X : [n] \to [n]$ is surjective onto $[\max(X)]$, $\overline{X}$ is its stable rank permutation, and logarithms are base two. Zero-frequency entropy terms contribute zero. The notation $X = X_1 \sqcup \cdots \sqcup X_k$ denotes a partition into disjoint, possibly interleaved subsequences. Heap measures are restricted to permutations. For brevity, increasing means non-decreasing, and decreasing means non-increasing.

Name (aliases)	Adaptive bound	Mathematical definition + intuition
Ham (Pos) [27]	$O(n + \mathrm{Ham}\log(\mathrm{Ham}+1))$	$\mathrm{Ham}(X) = \lvert\{i \in [n] \mid \overline{X}[i] \neq i\}\rvert$. *Number of elements outside their final positions.*
Exc [27]	$O(n + \mathrm{Exc}\log(\mathrm{Exc}+1))$	$\mathrm{Exc}(X) = n - c(X)$, where $c(X)$ is the number of cycles in $i \mapsto \overline{X}[i]$. *Minimum number of swaps to reach the stable sorted order; for distinct keys, the minimum number of swaps to sort X.*
Rem (Las) [13, 27]	$O(n + \mathrm{Rem}\log(\mathrm{Rem}+1))$	$\mathrm{Rem}(X) = \min\{\lvert Y\rvert : Y \subseteq X \text{ and } X - Y \text{ is increasing}\} = n - \mathrm{Las}(X)$. *Minimum number of deletions needed to leave X sorted.*
Max [27]	$O\big(n\big(1 + \log(\mathrm{Max}+1)\big)\big)$	$\mathrm{Max}(X) = \max_{i\in[n]} \lvert\overline{X}[i] - i\rvert$. *Maximum distance from an element's final position.*
Par (Dis, Radius) [2, 19]	$O\big(n\big(1 + \log(\mathrm{Par}+1)\big)\big)$	$\mathrm{Par}(X) = \max\{j - i : 1 \leq i < j \leq n,\ X[i] > X[j]\}$, with $\max\emptyset = 0$. *Maximum positional distance between an inverted pair.*
Inv [27]	$O\big(n + n\log\big(1 + \frac{\mathrm{Inv}}{n}\big)\big)$	$\mathrm{Inv}(X) = \lvert\{(i,j) : 1 \leq i < j \leq n,\ X[i] > X[j]\}\rvert$. *Number of inverted pairs.*
DS (Spear) [18]	$O\big(n + n\log\big(1 + \frac{\mathrm{DS}}{n}\big)\big)$	$\mathrm{DS}(X) = \sum_{i=1}^{n} \lvert\overline{X}[i] - i\rvert$. *Total distance from elements' final positions.*
HDS (HD) [4]	$O(n\,\mathrm{HDS})$	$\mathrm{HDS}(X) = \frac{1}{n}\sum_{i=1}^{n} \log\big(\lvert\overline{X}[i] - i\rvert + 2\big)$. *Average logarithmic distance from elements' final positions.*
nMulti [30]	$O(n\log(1 + \mathrm{nMulti}))$	$\mathrm{nMulti} = \max(X) = \lvert\{x \in X\}\rvert$. *Number of distinct values.*
HMulti [30]	$O(n(1 + \mathrm{HMulti}))$	$\mathrm{HMulti} = \sum_{i=1}^{n} \frac{m_i}{n}\log\frac{n}{m_i}$, where m_i is the number of occurrences of i in X. *Entropy of the value multiplicities.*
Block [10]	$O(n + \mathrm{Block}\log\mathrm{Block})$	$\mathrm{Block}(X) = 1 + \lvert\{i \in [n-1] : \overline{X}[i+1] \neq \overline{X}[i] + 1\}\rvert$. *Number of maximal sorted blocks contiguous in both position and value. The measure* nSRuns *uses the same count but the bound $O(n(1 + \log\mathrm{nSRuns}))$.*
HSRuns (HBlock) (H(vSRuns)) [6]	$O(n(1 + \mathrm{HSRuns}))$	$\mathrm{HSRuns}(X) = \mathrm{HBlock}(X) = \sum_{i=1}^{k} \frac{b_i}{n}\log\frac{n}{b_i}$, where $b_1, \ldots, b_k$ are the lengths of the maximal increasing blocks that are contiguous in both index and value. *Entropy of lengths of maximal sorted blocks contiguous in both position and value.*
nRuns [24, 27]	$O(n(1 + \log\mathrm{nRuns}))$	$\mathrm{nRuns}(X) = 1 + \lvert\{i \in [n-1] : X[i+1] < X[i]\}\rvert$. *Number of maximal sorted contiguous pieces.*
HRuns (H(vRuns), runlength) [6, 28]	$O(n(1 + \mathrm{HRuns}))$	$\mathrm{HRuns}(X) = \sum_{i=1}^{k} \frac{b_i}{n}\log\frac{n}{b_i}$, where $b_1, \ldots, b_k$ are the lengths of the maximal increasing index-contiguous blocks. *Entropy of lengths of maximal sorted contiguous pieces.*

Continued on next page

Name (aliases)	Adaptive bound	Mathematical definition + intuition
nRuns* (dual runs, riffles) [28]	$O(n(1+\log \mathrm{nRuns}^*))$	$\mathrm{nRuns}^*(X) = 1 + \lvert\{v \in [n-1] : \overline{X}^{-1}[v+1] < \overline{X}^{-1}[v]\}\rvert$. For permutations, this equals $\mathrm{nRuns}(X^{-1})$. *Number of maximal consecutive value ranges appearing in sorted order.*
HRuns* (dual runs, riffles) [20, 28]	$O(n(1+\mathrm{HRuns}^*))$	$\mathrm{HRuns}^*(X) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i}$, where $b_1, \ldots, b_k$ are the lengths of the maximal increasing value-contiguous subsequences. *Entropy of lengths of maximal consecutive value ranges appearing in sorted order.*
nSUS (SUS, Lds) [26]	$O(n(1+\log(\mathrm{nSUS})))$	$\mathrm{nSUS}(X) = \min\{k : X = X_1 \sqcup \cdots \sqcup X_k,\ X_r \text{ increasing}\}$. *Minimum number of increasing subsequences partitioning X.*
H(vSUS) (greedy SUS) [6]	$O\big(n\big(1+H(\mathrm{vSUS})\big)\big)$	$H(\mathrm{vSUS}(X)) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i}$, where $b_1, \ldots, b_k$ are the lengths of the increasing subsequences obtained by greedily scanning X left to right and appending each element to the subsequence with the largest last element at most the new element, starting a new subsequence if none exists. *Entropy of subsequence lengths in the greedy increasing partition.*
H(vLRM) [5]	$O\big(n\big(1+H(\mathrm{vLRM})\big)\big)$	$H(\mathrm{vLRM}(X)) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i}$, where $b_1, \ldots, b_k$ are the lengths of the subsequences obtained by repeatedly removing a longest root-to-leaf path from the remaining LRM forest, excluding the dummy root. *Entropy of subsequence lengths obtained by joining runs along the LRM tree.*
HSUS (chromatic entropy) [1, 6]	$O(n(1+\mathrm{HSUS}))$	$\mathrm{HSUS}(X) = \min_{X = X_1 \sqcup \cdots \sqcup X_k} \sum_{i=1}^{k} \frac{\lvert X_i\rvert}{n} \log \frac{n}{\lvert X_i\rvert}$, where each X_i is increasing. *Minimum entropy of subsequence lengths over increasing partitions.*
nSMS (SMS, Lads) [26, 36]	$O(n(1+\log(\mathrm{nSMS})))$	$\mathrm{nSMS}(X) = \min\{k : X = X_1 \sqcup \cdots \sqcup X_k,\ X_r \text{ increasing or decreasing}\}$. *Minimum number of monotone subsequences partitioning X.*
H(vSMS) [6]	$O\big(n\big(1+H(\mathrm{vSMS})\big)\big)$, given the partition	$H(\mathrm{vSMS}(X)) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i}$, where $b_i = \lvert X_i\rvert$ for a chosen partition $X = X_1 \sqcup \cdots \sqcup X_k$ into the minimum number of monotone subsequences. *Entropy of subsequence lengths in a specific monotone partition.*
HSMS [6]	$O(n(1+\mathrm{HSMS}))$	$\mathrm{HSMS}(X) = \min_{X = X_1 \sqcup \cdots \sqcup X_k} \sum_{i=1}^{k} \frac{\lvert X_i\rvert}{n} \log \frac{n}{\lvert X_i\rvert}$, with each X_i increasing or decreasing. *Minimum entropy of subsequence lengths over monotone partitions.*
enc [34]	$O(n(1+\log(\mathrm{enc})))$	$\mathrm{enc}(X) = k$, where k is the number of sorted lists built by scanning X left to right and adding each element at the beginning or end of the earliest-created list for which this preserves increasing order, creating a new list if none exists. *Number of sorted lists built greedily by insertion at either end.*
osc [25]	$O\big(n\big(1+\log\big(1+\frac{\mathrm{osc}}{n}\big)\big)\big)$	$\mathrm{osc}(X) = \sum_{i=1}^{n-1} (\max\{0, \lvert X[i+1] - X[i]\rvert - 1\})$. *Total number of distinct ranks strictly between consecutive values; the formula extends the permutation measure to repeated keys.*
Loc [23] (dynamic finger bound [11])	$O(n+\log(\mathrm{Loc}))$	$\mathrm{Loc}(X) = \prod_{i=2}^{n} \max\{1, \lvert \mathrm{rank}_{X[1..i]}(X[i]) - \mathrm{rank}_{X[1..i]}(X[i-1])\rvert\}$. *Consecutive elements should be close in rank among elements seen so far.*

Continued on next page

Name (aliases)	Adaptive bound	Mathematical definition + intuition
Hist [29] (working-set property [35])	$O(n + \log(\text{Hist}))$	$\text{Hist}(X) = \prod_{i=2}^{n} \min\left\{i-j : j < i \wedge \left\lvert \text{rank}_{X[1..i]}(X[i]) - \text{rank}_{X[1..i]}(X[j])\right\rvert \leq 1\right\}$. *Elements should have a recently inserted predecessor or successor.*
Reg [29,31] (unified bound [3,17])	$O(n + \log(\text{Reg}))$	$\text{Reg}(X) = \prod_{i=2}^{n} \min_{j<i}\left\{(i-j)\cdot \max\left\{1, \left\lvert \text{rank}_{X[1..i]}(X[i]) - \text{rank}_{X[1..i]}(X[j])\right\rvert\right\}\right\}$. *Elements should be close to an earlier element in position and current rank.*
loc [28]	$O(n + \log(\text{loc}))$	$\text{loc}(X) = \prod_{i=2}^{n} \max\{1, \lvert X[i] - X[i-1]\rvert\}$. *Consecutive elements should be close in final rank.*
hist [28]	$O(n + \log(\text{hist}))$	$\text{hist}(X) = \text{loc}(\overline{X}^{-1}) = \prod_{i=2}^{n} \lvert \overline{X}^{-1}[i] - \overline{X}^{-1}[i-1]\rvert$, *Elements of consecutive final ranks should be close in position.*
reg (this paper)	$O(n + \log(\text{reg}))$	$\text{reg}(X) = \prod_{i=2}^{n} \min_{j<i}\{(i-j)\max\{1, \lvert X[i] - X[j]\rvert\}\}$. *Each new element should be close to an earlier element in position and final rank.*
unimst (this paper)	$O(n + \text{unimst})$	$\text{unimst}(X)$ = weight of an MST with $w(i,j) = \log(2 + \lvert i-j\rvert + \lvert X[i] - X[j]\rvert)$. *Minimum spanning-tree weight for logarithmic distance in the index–rank plane.*
* (this paper)	$O(n + \log(^*))$	$^*(X^{-1}) = \prod_{i=2}^{n} \max\{1, \left\lvert \text{rank}_{X[i..n]}(X[i]) - \text{rank}_{X[i..n]}(X[i-1])\right\rvert\}$. *A variant of* Loc *for heapsort instead of insertion sort. X is a permutation.*
* (this paper)	$O(n + \log(^*))$	$^*(X^{-1}) = \prod_{i=2}^{n} \min\{n, \min\{i-j : j < i \wedge \lvert \underset{X[j..n]}{\text{rank}}(X[i]) - \underset{X[j..n]}{\text{rank}}(X[j])\rvert \leq 1\}\}$. *A variant of* Hist *for heapsort instead of insertion sort. X is a permutation.*
* (this paper) (unified bound [32])	$O(n + \log(^*))$	$^*(X^{-1}) = \prod_{i=2}^{n} \min_{j<i}\left\{(i-j)\cdot \max\left\{1, \left\lvert \underset{X[i..n]}{\text{rank}}(X[i]) - \underset{X[i..n]}{\text{rank}}(X[j])\right\rvert\right\}\right\}$. *A variant of* Reg *for heapsort instead of insertion sort. X is a permutation.*

C The Hierarchy of Adaptive Measures

Algorithm 6 SkeletonSort.

```
function SkeletonSort(S)
    if S is nil then
        return an empty list
    p ← the item returned by Random-Leaf(S)
    S^<, S^=, S^> ← Skeleton-Split(S, p)
    L_ℓ ← SkeletonSort(S^<)
    L_r ← SkeletonSort(S^>)
    return the concatenation of L_ℓ, the singleton list [p], and L_r
```

In Section 2.3, we show that reg $\preceq$ hist, reg $\preceq$ loc and reg $\preceq$ HSMS. In this section, we show the remaining relations in Figure 1, as depicted in **??**. Some of these relations were already shown by Moffat and Petersson [29], or Blelloch and Dobson [8]. While Moffat and Petersson use Hist and Loc, their proofs also work for hist and loc.

Many relations in **??** are straightforward from the definition. For example, nSUS considers partitions that are less restrictive than those of nRuns, which in turn is less restrictive than nSRuns; and all of these compute the same adaptive bound from a given permutation. Therefore nSUS $\preceq$ nRuns $\preceq$ nSRuns. Another example: HSUS minimises the entropy over a family of partitions that includes the one partition considered by $H(\text{vSUS})$. Therefore HSUS $\preceq H(\text{vSUS})$. Furthermore, as explained in the introduction, entropy-based measures are stronger than their counting variant, e.g. HRuns $\preceq$ nRuns. (See also Lemma 28 (1) below.) We explicitly prove the remaining relations in **??** in this section.

Inaccuracies in Barbay-Navarro Figure 1 in [6] claims the following relations, which seem to not hold: First they claim that Block $\cong$ nSRuns. However, these notions use different target running times: $O(n + \text{Block}\log(1 + \text{Block}))$ vs $O(n\log(1 + \text{nSRuns}))$. Second, they claim that $H(\text{vSUS}) \preceq H(\text{vLRM})$, but Section 4 and Lemma 7 in [5] show that these are incomparable. Third, they claim that $H(\text{vSMS}) \preceq H(\text{vSUS})$. This holds if $H(\text{vSMS})$ denotes the minimum entropy over all SMS partitions, but need not hold if vSMS is an unspecified minimum-cardinality partition, which may have much larger entropy than their greedy vSUS partition. What is true, however, is HSMS $\preceq$ HSUS.

C.1 Proving the remaining relations

We first prove a helper lemma on entropy functions.

▶ **Lemma 28.** *Let $b_1, \dots, b_k$ be positive integers with $b_1 + \cdots + b_k = n$. Let $H(b_1, \dots, b_k) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i}$ be the entropy function. Then*

$$nH(b_1, \dots, b_k) \leq n \log k \tag{1}$$

$$k \log n \leq 2n + k \log k \in O(n(1 + H(b_1, \dots, b_k))) \tag{2}$$

$$nH(b_1, \dots, b_k) \leq n + 2\sum_{i=1}^{k} b_i \log i \tag{3}$$

Proof. By concavity of log, we have

$$H(b_1, \dots, b_k) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i} \leq \log\Big(\sum_{i=1}^{k} \frac{b_i}{n} \cdot \frac{n}{b_i}\Big) = \log k.$$

This shows (1). Next, we show (2). By re-indexing, we may assume that $b_1 \geq b_2 \geq \cdots \geq b_k$. Then $\frac{n}{b_i} \geq i$ since $n \geq b_1 + \cdots + b_i \geq i \cdot b_i$. Also, $b_i \geq 1$ since b_i is positive. Thus,

$$nH(b_1, \dots, b_k) = \sum_{i=1}^{k} b_i \log \frac{n}{b_i} \geq \sum_{i=1}^{k} 1 \cdot \log i = \log \prod_{i=1}^{k} i = \log k! \in \Omega(k \log k).$$

In particular, $n + k \log k \in O(n(1 + H(b_1, \dots, b_k)))$. Finally, $\log \frac{n}{k} \leq \log e \cdot \frac{n}{k}$ and thus

$$k \log n = (k \log k + k \log \frac{n}{k}) \leq k \log k + k \log e \cdot \frac{n}{k} \leq k \log k + 2n.$$

This shows (2). Next, we show (3). The log-sum inequality for sequences $y_1, \dots, y_k > 0$ and $z_1, \dots, z_k > 0$ with $Y = y_1 + \cdots + y_k$ and $Z = z_1 + \cdots + z_k$ states that

$$\sum_{i=1}^{k} y_i \log \frac{1}{y_i} \leq \sum_{i=1}^{k} y_i \log \frac{1}{z_i} + Y \log \frac{Z}{Y}.$$

We apply this with $y_i := \frac{b_i}{n}$ and $z_i := \frac{1}{i^2}$, which yields $Y = 1$, $Z \leq \frac{\pi^2}{6}$, and

$$H(b_1, \dots, b_k) = \sum_{i=1}^{k} \frac{b_i}{n} \log \frac{n}{b_i} \leq \sum_{i=1}^{k} \frac{b_i}{n} \log(i^2) + \log \frac{\pi^2}{6} \leq 2\sum_{i=1}^{k} \frac{b_i}{n} \log i + 1.$$

This shows (3). ◀

Lemma 28 (1) immediately implies the "entropy" arrows in **??**.

▶ **Lemma 29.** Block $\preceq$ HSRuns.

Proof. Let $X = X_1 \sqcup \cdots \sqcup X_k$ be the partition of X into maximal increasing sequences that are both contiguous in index and value. By Lemma 28 (2),

$$n{+}\mathrm{Block}\log(1{+}\mathrm{Block}) = n{+}k\log(k{+}1) \in O(n(1{+}H(|X_1|,\dots,|X_k|))) = O(n(1{+}\mathrm{HSRuns})).$$

This shows Block $\preceq$ HSRuns. ◀

▶ **Lemma 30.** HDS $\preceq$ DS.

Proof. By concavity of $x \mapsto \log(2+x)$, we have

$$\mathrm{HDS} = \sum_{i=1}^{n} \frac{\log(2+|\overline{X}[i]-i|)}{n} \le \log(2+\frac{\sum_{i=1}^{n}|\overline{X}[i]-i|}{n}) = \log(2+\frac{\mathrm{DS}}{n}).$$

In particular, $n\,\mathrm{HDS} \in O(n + n\log(1+\frac{\mathrm{DS}}{n}))$. This shows HDS $\preceq$ DS. ◀

▶ **Lemma 31.** loc $\preceq$ HRuns *and* hist $\preceq$ HRuns*.

Proof. Let $X = X_1 \sqcup \cdots \sqcup X_k$ be the partition of X into maximal increasing sequences that are contiguous in index. Put $s_i = |X_i|$. A singleton run contributes zero. For $s_i > 1$, monotonicity gives $\sum_{j=2}^{s_i} \max\{1, X_i[j]-X_i[j-1]\} \le s_i - 1 + n$. By concavity of log,

$$\sum_{j=2}^{s_i} \log\max\{1,|X_i[j]-X_i[j-1]|\} \le (s_i-1)\log\left(1+\frac{n}{s_i-1}\right) \le s_i\log\frac{2n}{s_i}.$$

Due to the boundaries between the X_i, the sum $\log\mathrm{loc}$ contains $k-1$ additional terms that are each at most $\log n$. Hence, by Lemma 28 (2), we have

$$\log\mathrm{loc} \le k\log n + \sum_{i=1}^{k} s_i\log\frac{2n}{s_i} = k\log n + n + \sum_{i=1}^{k} s_i\log\frac{n}{s_i} \in O(n(1+H(s_1,\dots,s_k))).$$

This shows that loc $\preceq$ HRuns. For hist, observe that $\mathrm{hist}(X) = \mathrm{loc}(\overline{X}^{-1})$. On the other hand, $\mathrm{HRuns}^*(\overline{X}) \le \mathrm{HRuns}^*(X)$. Indeed, any value-contiguous subsequence of X contains, for each value, either all copies or no copies of that value. Thus, a value-contiguous subsequence of X is also a value-contiguous subsequence of $\overline{X}$. The partition of $\overline{X}$ into maximal such subsequences therefore only merges parts, and entropy does not increase under merging. Applying loc $\preceq$ HRuns to $\overline{X}^{-1}$ now shows that hist $\preceq$ HRuns*. ◀

▶ **Lemma 32.** loc $\preceq$ HDS *and* hist $\preceq$ HDS.

Proof. For positive integers a, b, c, by the triangle inequality, we have

$$2 + |c-a| \le 3 + |a-b| + |c-(b+1)| \le (2+|a-b|)\cdot(2+|c-(b+1)|).$$

We use this with $a = \overline{X}[i-1], b = i-1, c = \overline{X}[i]$, and use the fact that $|X[i]-X[i-1]| \le |\overline{X}[i]-\overline{X}[i-1]|$ to get

$$\mathrm{loc} \le \prod_{i=2}^{n}(2{+}|\overline{X}[i]{-}\overline{X}[i{-}1]|) \le \prod_{i=2}^{n}\Big((2{+}|\overline{X}[i{-}1]{-}(i{-}1)|)\cdot(2{+}|\overline{X}[i]{-}i|)\Big) \le \prod_{i=1}^{n}(2{+}|\overline{X}[i]{-}i|)^2.$$

Taking logarithms yields $\log\mathrm{loc} \le 2n\,\mathrm{HDS}$, which shows loc $\preceq$ HDS. Since $\mathrm{hist}(X) = \mathrm{loc}(\overline{X}^{-1})$ and $\mathrm{HDS}(X) = \mathrm{HDS}(\overline{X}) = \mathrm{HDS}(\overline{X}^{-1})$, this also shows hist $\preceq$ HDS. ◀

▶ **Lemma 33.** HSUS $\preceq$ HDS.

Proof. Apply the following construction to the permutation $P := \overline{X}$. Any increasing subsequence of P corresponds to a non-decreasing subsequence of X, and $\mathrm{HDS}(P) = \mathrm{HDS}(X)$. For each integer d, let Y_d be the subsequence consisting of entries $P[i]$ with $P[i]-i = d$. Each Y_d is increasing. Order these subsequences as $Y_0, Y_1, Y_{-1}, Y_2, Y_{-2}, \dots$ and discard empty parts, obtaining $X_1, \dots, X_k$. An entry $P[i] \in X_j$ then satisfies $j \le 2(2 + |P[i] - i|)$. By Lemma 28 (3),

$$nH(|X_1|, \dots, |X_k|) \le n + 2\sum_{j=1}^{k} |X_j| \log j \le n + 2\sum_{i=1}^{n} \log(2(2+|P[i]-i|)) \in O(n(1+\mathrm{HDS}(X))).$$

Transferring this partition to X proves HSUS $\preceq$ HDS. ◀

▶ **Lemma 34.** HSUS $\preceq$ Rem.

Proof. Let $k = \mathrm{Rem} + 1$. We partition $X = X_1 \sqcup \cdots \sqcup X_k$ where $X_2, \dots, X_k$ are singletons whose removal leaves the sorted sequence X_1. By Lemma 28 (2),

$$nH(|X_1|, \dots, |X_k|) = |X_1| \log \frac{n}{|X_1|} + \sum_{i=2}^{k} \log n \le 2n + k \log n \le 4n + k \log k.$$

Thus, $n(1 + \mathrm{HSUS}) \in O(n + \mathrm{Rem} \log(1 + \mathrm{Rem}))$, which shows HSUS $\preceq$ Rem. ◀

▶ **Lemma 35.** Loc $\preceq$ loc *and* * $\preceq$ hist. *Similarly,* Reg $\preceq$ reg *and, on permutations,* * $\preceq$ reg.

Proof. We have $\left|\mathrm{rank}_{X[1..i]}(X[i]) - \mathrm{rank}_{X[1..i]}(X[i-1])\right| \le |X[i] - X[i-1]|$, and thus

$$\begin{aligned}\mathrm{Loc}(X) &= \prod_{i=2}^{n} \max\{1, |\mathrm{rank}_{X[1..i]}(X[i]) - \mathrm{rank}_{X[1..i]}(X[i-1])|\} \\ &\le \prod_{i=2}^{n} \max\{1, |X[i] - X[i-1]|\} = \mathrm{loc}(X).\end{aligned}$$

This shows Loc $\preceq$ loc. Similarly, $\left|\mathrm{rank}_{X[i..n]}(X[i]) - \mathrm{rank}_{X[i..n]}(X[i-1])\right| \le |X[i] - X[i-1]|$, so the same argument shows $\preceq$ loc. * is only defined when X is a permutation, in which case $^*(X) = (X^{-1})$, while also $\mathrm{hist}(X) = \mathrm{loc}(X^{-1})$. Thus, $\preceq$ loc implies * $\preceq$ hist.

The same rank-distance argument gives Reg $\preceq$ reg and $\preceq$ reg. By Theorem 5, $n + \log \mathrm{reg}(X^{-1}) \in \Theta(n + \log \mathrm{reg}(X))$ for permutations, so $\preceq$ reg implies * $\preceq$ reg. ◀

▶ **Lemma 36.** loc $\preceq$ nMulti.

Proof. We have

$$\mathrm{loc}(X) = \prod_{i=2}^{n} \max\{1, |X[i] - X[i-1]|\} \le \max(X)^{n-1} \le \max(X)^n.$$

Taking logarithms yields $n + \log \mathrm{loc} \in O(n \log(1 + \mathrm{nMulti}))$, which shows loc $\preceq$ nMulti. ◀

▶ **Lemma 37.** Hist $\preceq$ hist.

Proof. If we replace X by $\overline{X}$, then Hist only increases, while hist is unaffected. Thus, it suffices to consider the cases where X is a permutation.

For $i \in [n]$, let $S_i := \{j \in [n] : j < i \wedge |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[j])| \leq 1\}$. Our goal is to define a function $f : [n] \setminus \{1\} \to [n]$ with $f(i) \in S_i$ for all i and $\sum_{i=2}^{n} \log(i - f(i)) \in O(n + \log \operatorname{hist}(X))$. Given such a function, picking $j = f(i)$ in the definition of Hist shows Hist $\preceq$ hist.

For $t \in [n]$, the *t-front* consists of all $i \in [n]$ with $X[i] \leq t$ such that there exists no $j < i$ with $X[i] \leq X[j] \leq t$.

▷ Claim 38. Let $t \in [n]$ be arbitrary. Put $I_t = X^{-1}([1..t])$ and $r_t = \min(I_t)$. There exists a function $f : I_t \setminus \{r_t\} \to I_t$ with $f(i) \in S_i$ for all i with the sum property

$$\sum_{i \in I_t \setminus \{r_t\}} \log(i - f(i)) \leq \sum_{y=2}^{t} \log |X^{-1}[y] - X^{-1}[y-1]|,$$

and the following additional property: If $i_1 < \cdots < i_k$ are the indices of the t-front, then $f(i_j) = i_{j-1}$ for all $j \geq 2$.

Proof of the Claim. We do induction over t. The base case $t = 1$ is straightforward as f is the trivial function and both sums in the sum property are empty. Let $t \geq 2$ be arbitrary. Suppose the claim holds for $t-1$ and yields a function f_{t-1}. We construct f_t by slightly modifying f_{t-1}, keeping all choices unchanged unless stated otherwise. Let $i_1, \ldots, i_k$ be the $(t-1)$-front. Put $i = X^{-1}[t]$. If $i_k < i$, then the t-front is $i_1, \ldots, i_k, i$. Put $f_t(i) = i_k$. The sum property is maintained since $i_k = X^{-1}[t-1]$ and $i = X^{-1}[t]$, meaning $\log(i - f_t(i)) = \log |X^{-1}[t] - X^{-1}[t-1]|$.

Otherwise, suppose that $i_j < i < i_{j+1}$ for some index j. We put $f_t(i) = i_j$ and $f_t(i_{j+1}) = i$, and keep $f_t(i') = f_{t-1}(i')$ for all other arguments i'. The t-front is $i_1, \ldots, i_j, i$, so f_t satisfies the additional property. We have

$$i - f_t(i) = i - i_j \leq i_{j+1} - i_j = i_{j+1} - f_{t-1}(i_{j+1})$$

and

$$i_{j+1} - f_t(i_{j+1}) = i_{j+1} - i \leq i_k - i = X^{-1}[t-1] - X^{-1}[t].$$

Therefore, the sum property for $(t-1)$ implies the sum-property for t:

$$\begin{aligned}
&\sum_{i' \in I_t \setminus \{r_t\}} \log(i' - f_t(i')) \\
&= \log(i - f_t(i)) + \log(i_{j+1} - f_t(i_{j+1})) - \log(i_{j+1} - f_{t-1}(i_{j+1})) + \sum_{i' \in I_{t-1} \setminus \{r_{t-1}\}} \log(i' - f_{t-1}(i')) \\
&\leq \log |X^{-1}[t-1] - X^{-1}[t]| + \sum_{y=2}^{t-1} \log |X^{-1}[y] - X^{-1}[y-1]| \\
&= \sum_{y=2}^{t} \log |X^{-1}[y] - X^{-1}[y-1]|.
\end{aligned}$$

Finally, if $i < i_1$, then $i_1 = r_{t-1}$ and $i = r_t$. We put $f_t(i_1) = i$. The sum property follows since

$$\log(i_1 - i) \leq \log(i_k - i) = \log |X^{-1}[t-1] - X^{-1}[t]|.$$

In all three cases, the choices belong to the corresponding sets S_i: the elements of the $(t-1)$-front are prefix maxima among the values $1,\dots,t-1$, and t is larger than all these values. ◀

The claim with $t=n$ immediately yields the desired function f. This shows Hist $\preceq$ hist. ◀

▶ **Lemma 39.** $^* \preceq$ HRuns.

Proof. We instead show $\preceq$ HRuns*. Since heap measures are defined only on permutations, assume that X is a permutation. Let $X_1,\dots,X_k$ be the partition of X into maximal value-contiguous increasing sequences. Put $s_i := |X_i|$. Write a single part as $X_i = (X[j_1],\dots,X[j_{s_i}])$, where $j_1 < \cdots < j_{s_i}$. Since this part is value-contiguous and increasing, $|\operatorname{rank}_X(X[j_\ell]) - \operatorname{rank}_X(X[j_{\ell-1}])| = 1$ for $\ell \geq 2$. Hence

$$\prod_{\ell=2}^{s_i} \min\{j_\ell - j : j < j_\ell \wedge |\operatorname{rank}_{X[j..n]}(X[j_\ell]) - \operatorname{rank}_{X[j..n]}(X[j])| \leq 1\} \leq \prod_{\ell=2}^{s_i}(j_\ell - j_{\ell-1}).$$

As in the proof of Lemma 31, concavity of the logarithm bounds this product by $(4n/s_i)^{s_i}$; for $s_i = 1$, the product is empty and equals one. We plug this into the definition of , and pay a factor n for the first element j_1 in each sequence X_i:

$$\begin{aligned}(X) &= \prod_{i'=2}^{n} \min\{n, \min\{i'-j : j < i' \wedge |\operatorname{rank}_{X[j..n]}(X[i']) - \operatorname{rank}_{X[j..n]}(X[j])| \leq 1\}\} \\ &\leq n^k \prod_{i=1}^{k} \Big(\frac{4n}{s_i}\Big)^{s_i}.\end{aligned}$$

Taking logarithms gives $\log(X) \in O(n + k\log n + nH(s_1,\dots,s_k))$. By Lemma 28 (2), this is $O(n(1+H(s_1,\dots,s_k))) = O(n(1+\text{HRuns}^*(X)))$.

This shows $\preceq$ HRuns*. Taking the inverse permutation shows $^* \preceq$ HRuns. ◀

C.2 Reg equals reg for permutations

Lemma 35 shows Reg $\preceq$ reg. In Section 2.1 we show that, if X contains duplicates, then reg $\not\preceq$ Reg. However, if X is a permutation, then we can show that reg $\preceq$ Reg.

▶ **Theorem 11.** *If X is a permutation, then* $\log \text{reg} \in O(n + \log \text{Reg})$.

▶ **Lemma 40.** *Let X be a permutation. For $i \geq 2$, put*

$$\begin{aligned} d(i,j) &= |X[i] - X[j]| \\ D(i,j) &= |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..j]}(X[j])| \\ r(i) &:= \min_{j<i} \max\{(i-j), d(i,j)\} \\ R(i) &:= \min_{j<i} \max\{(i-j), D(i,j)\}. \end{aligned}$$

For every integer $t \geq 1$, we have

$$|\{i : r(i) \geq t^3\}| \leq 5|\{i : R(i) \geq t\}| + O\big(\frac{n}{t}\big).$$

Proof of Theorem 11 from Lemma 40. We apply Lemma 40 for $t = 2^j$ where $0 \le j \le \lfloor \log n \rfloor$. This yields

$$\underbrace{\sum_{j=0}^{\lfloor \log n \rfloor} |\{i : r(i) \ge 2^{3j}\}|}_{=:S} \le 5 \underbrace{\sum_{j=0}^{\lfloor \log n \rfloor} |\{i : R(i) \ge 2^j\}|}_{=:T} + O(n).$$

The contribution of an index i to S is at least $\log(r(i))/3 - 1$, and the contribution to T is at most $\log(R(i)) + 1$. Therefore, this shows

$$\sum_{i=2}^{n} \log r(i) \le 3S + O(n) \le 15T + O(n) \le 30 \sum_{i=2}^{n} \log R(i) + O(n).$$

We have $\log \mathrm{reg} \le 2 \sum_{i=2}^{n} \log r(i)$. We also have $\log \mathrm{Reg} \ge \sum_{i=2}^{n} \log R(i)$. Indeed, for $j < i$, put $a = i - j$ and $b = |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[j])|$. Since X is a permutation, $b \ge 1$. The rank of $X[j]$ can increase by at most a between steps j and i. If $X[i] > X[j]$, at most $a - 1$ of these elements are smaller than $X[j]$, so $D(i, j) \le b + a - 1$. If $X[i] < X[j]$, the rank of $X[j]$ increases by at least one, and again $D(i, j) \le b + a - 1$. Thus, $D(i, j) \le a + b - 1 \le ab$. In particular, $R(i) \le ab$ for every $j < i$, proving the claimed inequality. Therefore, $\log \mathrm{reg} \le 60 \log \mathrm{Reg} + O(n)$. ◀

Proof of Lemma 40. The claim is immediate for $t = 1$ or $t^3 > n$, so assume $2 \le t \le n^{1/3}$. Put $F(i) := \arg\min_{j<i} \max\{(i - j), D(i, j)\}$. We partition the value space $[n]$ into *buckets* of size t^3. The last bucket may be smaller. The item $X[i]$ goes into the bucket $\lfloor (X[i] - 1)/t^3 \rfloor$. We say an index i is *early* if $X[i]$ is among the first $4t^2$ items inserted in its bucket. Observe that the total number of early items is $O(\frac{n}{t^3} \cdot 4t^2) = O(\frac{n}{t})$.

Additionally, we partition the index space into *epochs* of size t^2. The last epoch may be smaller. The item $X[i]$ goes into the epoch $\lfloor (i - 1)/t^2 \rfloor$. We say a bucket is *heavy* at the start of an epoch if it contains at least $2t^2$ previously inserted items, and *light* otherwise.

Our proof is a charging argument where we assign one charge for every item i with $r(i) \ge t^3$. We are allowed to assign these charges as follows: At most 5 charges to every item i with $R(i) \ge t$, at most 5 charges to every early item, and at most 5 charges to every bucket-epoch intersection that contains at least $t/3$ items. This budget guarantees that the total number of charges is at most $5|\{i : R(i) \ge t\}| + O\big(\frac{n}{t}\big)$, which shows the lemma.

Let $i \in [n]$ be an index with $r(i) \ge t^3$. If i is early, then we charge i to itself. Otherwise, let E be the epoch of i and $E - 1$ the previous epoch. Since i is not early, at least $4t^2$ items were inserted into its bucket B before i. At most $2t^2$ of these were inserted during epochs $E - 1$ and E, so B is $(E - 1)$-heavy. If $R(i) \ge t$, then we charge i to itself. Otherwise, consider the *finger path* $j_1 = F(i), j_2 = F(j_1), \ldots$ until we hit the first item j_a that is early, has $R(j_a) \ge t$, or until we reach j_a for $a = t$. This finger path is very well-behaved: We have $i > j_1 > \cdots > j_a$, and also $j_a > i - t^2$ since $R(i), R(j_1), \ldots, R(j_{a-1}) < t$, and $a \le t$. In particular, the finger path lies within the epochs E and $(E - 1)$, and, since $r(i) \ge t^3$, it is disjoint from bucket B.

Also, the finger path can never cross over an $(E - 1)$-heavy bucket. To see this, put $\rho(j) := \operatorname{rank}_{X[1..j]}(X[j])$. For any two vertices p, q on the finger path, including i, we have

$$|p - q| < t^2 \qquad \text{and} \qquad |\rho(p) - \rho(q)| < t^2.$$

Suppose that $X[p]$ and $X[q]$ lie on opposite sides of an $(E - 1)$-heavy bucket C. At least $2t^2$ elements of C were inserted before epoch $E - 1$. Since the insertion ranks can change by at most $|p - q|$ between steps p and q, we have $|\rho(p) - \rho(q)| \ge 2t^2 - |p - q| \ge t^2$, a contradiction.

Since $j_1, \ldots, j_{a-1}$ are not early, their buckets are all $(E-1)$-heavy. Thus, if $a > 1$, the barrier argument shows that $j_1, \ldots, j_{a-1}$ lie in the same $(E-1)$-heavy bucket B' adjacent to B among the $(E-1)$-heavy buckets, and j_a lies in B' or in an $(E-1)$-light bucket between B and B'. If $a = 1$, then j_a lies in a neighbouring $(E-1)$-heavy bucket or in a light bucket between B and the next heavy bucket.

If $a < t$, then we charge i to j_a. Otherwise, one of the pairs (B', E) and $(B', E-1)$ contains at least half of $j_1, \ldots, j_{t-1}$, and thus at least $t/3$ items, so we charge i to that pair.

To analyse the number of charges assigned to each item or pair, observe that every bucket-epoch pair contains at most one index $i \in [n]$ with $r(i) \geq t^3$. Indeed, two items in the same bucket and epoch have both index distance less than t^2 and value distance less than t^3.

Consider an item j that receives a charge as the endpoint of a finger path. If j lies in epoch K, the source item i lies in epoch K or $K+1$. For each of these two possibilities, the source bucket B must be one of the two $(E-1)$-heavy buckets neighbouring the bucket of j, or the light interval containing it. Since each bucket-epoch pair contains at most one source item, j receives at most four such charges, plus possibly one charge from itself.

Similarly, consider a bucket-epoch pair (B', K) that receives a charge. The source item lies in epoch K or $K+1$, and its bucket is one of the two neighbouring heavy buckets of B' at the start of epoch $E-1$. Thus, this pair receives at most four charges.

Finally, there are $O(n/t)$ early items and $O(n/t)$ bucket-epoch pairs containing at least $t/3$ items. Therefore, the total number of charges is at most $5|\{i : R(i) \geq t\}| + O(n/t)$, proving the lemma. ◀

D Loc is the Dynamic Finger Property, and Hist is the Working Set

In Section 2, we showed that insertion sort using a dictionary with the unified bound is Reg-adaptive. In this section, we similarly relate Loc and Hist to the dynamic-finger and working-set properties of binary search trees [11, 35]. Using a BST with the dynamic-finger property as a heap yields a *-adaptive algorithm. We also show that a BST with an adjacent-working-set property would yield a *-adaptive heap algorithm; the latter result is conditional. Similar to reg, this suggests that loc and hist are perhaps the better universal definitions.

Recall that a binary search tree stores a sorted set Y subject to an ACCESS(y) operation that forces it to navigate from the root to some item $y \in Y$. A BST has the *working-set property* if ACCESS(y) takes amortised $O(\log(2 + \text{age}(y)))$ time, where $\text{age}(y)$ counts the distinct keys accessed since the previous access to y. For a key not previously accessed, use $\text{age}(y) = |Y|$. Counting all intervening access operations gives an upper bound on this age, which suffices in the proofs below. Splay trees have the working-set property [35]. A BST has the *dynamic-finger property* [11] if ACCESS(y) takes amortised $O(\log(2 + d(y', y)))$ time, where y' is the previously accessed item and $d(y', y)$ is the number of items in Y that lie between y and y'. Cole [11], building on joint work with Mishra, Schmidt, and Siegel [12], proves the dynamic-finger property for splay trees.

There are different algorithms for inserting an item y' into a binary search tree. For this section, we allow any insertion algorithm that accesses both the predecessor and successor of y' in Y, adds a new node y', and then accesses y'. To perform insertion sort, we start with a binary search tree that contains two dummy elements $-\infty, \infty$, and then insert $X[1], \ldots, X[n]$ in this order.

▶ **Theorem 41.** *Insertion sort with a binary search tree that has the dynamic finger property is* Loc*-adaptive.*

Proof. Inserting $X[1]$ takes constant time. For $i \geq 2$, when inserting $y = X[i]$, the last accessed item was $y' = X[i-1]$. Let z be the predecessor or successor of y accessed first. Then $d(y', z) \leq |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[i-1])| + 1$, while the remaining accesses during insertion take constant amortised time. Thus, the total running time is

$$O\Big(\sum_{i=2}^{n}(2 + \log(3 + |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[i-1])|)\Big)$$

$$= O\Big(n + \sum_{i=2}^{n}\log(\max\{1, |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[i-1])|\})\Big) = O(n + \log \mathrm{Loc}). \blacktriangleleft$$

▶ **Theorem 42.** *Insertion sort with a binary search tree that has the working set property is* Hist*-adaptive.*

Proof. Inserting $X[1]$ takes constant time. For $i \geq 2$, when inserting $y = X[i]$ into Y, let $j = \arg\min\{i - j : j < i \wedge |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[j])| \leq 1\}$. Then $y' := X[j]$ is the predecessor or successor of $X[i]$ in Y, and the other predecessor or successor of $X[i]$ is an element y'' that was inserted before $X[j]$. Hence, the insertion of y' accessed both y' and y''. Since every insertion accesses three nodes, this shows $\operatorname{age}(y'), \operatorname{age}(y'') \leq 3(i-j)$, so that accessing y'' and y' during the insertion of y takes $O(1 + \log(i-j))$ time. We assume that accessing the node $X[i]$ that was just created is free. Then, the total running time is

$$O\left(n + \sum_{i=2}^{n}\log\min\{i - j : j < i \wedge |\operatorname{rank}_{X[1..i]}(X[i]) - \operatorname{rank}_{X[1..i]}(X[j])| \leq 1\}\right) = O(n + \log \mathrm{Hist}(X)). \blacktriangleleft$$

D.1 Heap versions of Loc and Hist

Recall that heapsort repeatedly removes and outputs the minimum remaining element of X. If X contains duplicates, the minimum value is still well defined, but the extraction order of equal occurrences requires a tie-breaking rule for the inverse-permutation analysis. To avoid this issue, we assume that X is a permutation in this section.

If we compare the definitions of Reg and , then the only difference is that $\operatorname{rank}_{X[1..i]}$ is replaced by $\operatorname{rank}_{X[i..n]}$. The corresponding heap measures also use suffix ranks. For , the suffix begins at the current index i; for , it begins at the earlier index j, when the finger was extracted:

$$(X) = \prod_{i=2}^{n}\max\{1, \left|\operatorname{rank}_{X[i..n]}(X[i]) - \operatorname{rank}_{X[i..n]}(X[i-1])\right|\}$$

$$(X) = \prod_{i=2}^{n}\min\{n, \min\{i - j : j < i \wedge |\operatorname*{rank}_{X[j..n]}(X[i]) - \operatorname*{rank}_{X[j..n]}(X[j])| \leq 1\}\}$$

We then put $^*(X) := (X^{-1})$ and $^*(X) := (X^{-1})$. The measure * behaves as expected and $^* \preceq$ hist. In contrast, * does not dominate loc.

Heapsort For simplicity, we assume that a heap is initialised on some list in linear time and only needs to support ExtractMin. This interface avoids having to support Insert, and is sufficient to perform heapsort. There is a standard way of using a binary search tree as a heap: Build a binary search tree with nodes $1, \ldots, n$, and at every node u, store the index of the node i in the subtree of u that minimises $X[i]$. To perform ExtractMin, find the index

i of smallest element at the root, and then delete the node i. Similar to insertion, there are different algorithms for deleting an item y from a binary search tree. For this section, we use a deletion routine that accesses the deleted node and its predecessor and successor, performs $O(1)$ additional structural changes, and ends at a surviving neighbour. Missing neighbours are represented by sentinel keys. The access bounds are assumed to apply to the resulting update sequence.

If we use a binary search tree with the dynamic finger property, then we obtain a heap with a "dual dynamic finger property". As far as we know, such heaps have not been studied before. We show:

▶ **Theorem 43.** *Heapsort that uses a binary search tree with the dynamic finger property as a heap is *-adaptive.*

Proof. Since X is a permutation, heapsort will output the values $1, 2, \ldots, n$ in this order, and these elements occur in X at the indices $X^{-1}[1], \ldots, X^{-1}[n]$. The *value* of a node i refers to $X[i]$. To analyse the running time of the EXTRACTMIN operation that returns some value y, let $u_y := X^{-1}[y]$ be the node that contains this value, and define u_{y-1} similarly. Consider the state of the binary search tree during the previous EXTRACTMIN operation, right before the node with value $y-1$ is deleted. At this time, the binary search tree contains the values $y-1, \ldots, n$, so the number of nodes in the binary search tree between u_y and u_{y-1} is

$$\begin{aligned} d(u_y, u_{y-1}) &= |\{k \geq y-1 : \min(X^{-1}[y], X^{-1}[y-1]) \leq X^{-1}[k] \leq \max(X^{-1}[y], X^{-1}[y-1])\}| \\ &= |\operatorname{rank}_{X^{-1}[(y-1)..n]}(X^{-1}[y]) - \operatorname{rank}_{X^{-1}[(y-1)..n]}(X^{-1}[y-1])| + 1 \\ &\leq |\operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[y]) - \operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[y-1])| + 2, \end{aligned}$$

Deleting $y-1$ only decreases $d(u_y, u_{y-1})$. The last accessed node during the deletion of u_{y-1} is the predecessor or successor of u_{y-1}, and the first accessed node during the deletion of u_y is either u_y or its predecessor or successor. Hence, by the dynamic finger property, the first access during the deletion of u_y takes $O(\log(2 + d(u_{y-1}, u_y)))$ time, and subsequent accesses take $O(1)$ time. After summing over all y, the total running time is

$$\begin{aligned} &O\Big(n + \sum_{y=2}^{n} \log(2 + d(u_y, u_{y-1}))\Big) \\ &= O\Big(n + \sum_{y=2}^{n} \log(2 + |\operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[y]) - \operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[y-1])|)\Big), \end{aligned}$$

which is $O(n + \log^*(X))$. ◀

Obtaining an adaptive heap from the working set property of binary search trees is more difficult. The issue is that the age of a node u says nothing about the age of the predecessor and successor of u. We propose the *adjacent working set property* for binary search trees, which allows you to not only access a node u, but also its predecessor and successor, in $O(\log(2 + \operatorname{age}(u)))$ time, independent of the age of this predecessor or successor. We are not aware of any binary search tree with this property, but we observe that such a hypothetical binary search tree would yield an interesting heap with a "dual adjacent working set property". The *working-set property* for heaps introduced by Iacono [21] is a different notion: under this BST-to-heap interpretation, it corresponds to a static-finger property rather than the BST working-set property.

▶ **Theorem 44.** *Heapsort that uses a binary search tree with the adjacent working set property as a heap is *-adaptive.*

Proof. Let $y < x$ be values stored at the nodes $u_x = X^{-1}[x]$ and $u_y = X^{-1}[y]$. If $|\operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[x]) - \operatorname{rank}_{X^{-1}[y..n]}(X^{-1}[y])| \le 1$, then right before node u_y is deleted, u_x and u_y are consecutive nodes in the binary search tree. In particular, deleting u_y accesses u_x. Subsequently, the nodes $u_{y+1}, \dots, u_x$ are deleted in this order. Since each deletion accesses only three nodes, at the time when u_x is deleted, we have $\operatorname{age}(u_x) \le 3(x-y)$, so this deletion takes $O(\log(2+x-y))$ amortised time. Since x, y were arbitrary, for each x, we may take the minimum over all y. If no suitable y exists, the cost of the deletion is $O(\log n)$. Hence, the total running time is

$$\Big(\sum_{x=1}^{n} \log(2 + \min\{n, \min\{x-y : y < x \wedge |\operatorname*{rank}_{X^{-1}[y..n]}(X^{-1}[x]) - \operatorname*{rank}_{X^{-1}[y..n]}(X^{-1}[y])| \le 1\}\})\Big),$$

which is $O(n + \log^*(X))$. ◀

The measure * behaves as expected: Lemma 35 shows $^* \preceq$ hist. However, the measure * has strange properties: The sequence $3, 2, 1, 6, 5, 4, \dots, n, n-1, n-2$ has $\log \in \Theta(n \log n)$, but $n \log \mathrm{Par} \in O(n)$. This permutation is self-inverse and has bounded loc cost, so it shows both $^* \not\preceq \mathrm{Par}$ and $^* \not\preceq \mathrm{loc}$.

E Weighted reg + loc, Synergistic Sorting

Barbay, Ochoa and Satti [7] introduce two sorting algorithms DLMSort and QuickSynergySort that combine many adaptive properties. We show that their notion of adaptivity is stronger than nRuns, Block, and HMulti, but weaker than a weighted version of loc. To this end, we define the measure Syn that describes their algorithm.

▶ **Definition 45.** *Let $X_1, \dots, X_k$ be the partition of X into maximal contiguous non-decreasing sequences. We say a value $x \in [n]$ is* free *if, for every index j with $X[j] = x$, we have $\overline{X}[j] = \overline{X}[j-1]+1$ and $\overline{X}[j] = \overline{X}[j+1]-1$. Note that, if x is free, then x occurs in at most one X_i. For every non-free value x, let m_x be the number of X_i that contain at least one copy of x. Let* $\mathrm{Syn} = \prod_{x:\ non\text{-}free} \binom{k}{m_x}$ *and* $f_{\mathrm{Syn}}(X) = n + \log \mathrm{Syn}(X)$.

▶ **Theorem 46** (Corollary 1 and Theorem 5 in [7])**.** *DLMSort and QuickSynergySort are* Syn*-adaptive.*

▶ **Lemma 47.** $\mathrm{Syn} \preceq \mathrm{nRuns}$, $\mathrm{Syn} \preceq \mathrm{HMulti}$ *and* $\mathrm{Syn} \preceq \mathrm{Block}$.

Proof. We use the notation of Definition 45. Since $\binom{k}{i} \le \left(\frac{ek}{k}\right)^k$, we have

$$\log \mathrm{Syn} = \sum_{x:\ \text{non-free}} \log \binom{k}{m_x} \le \sum_{x:\ \text{non-free}} m_x \log \frac{ek}{m_x} \le \sum_{x:\ \text{non-free}} m_x \log(ek) \le n \log(ek).$$

Since $k = \mathrm{nRuns}$, this shows $n + \log \mathrm{Syn}(X) \in O(n + n \log \mathrm{nRuns})$, i.e. $\mathrm{Syn} \preceq \mathrm{nRuns}$.

For every value $x \in [n]$, let m'_x denote the number of occurrences of x in X. Since $y \mapsto y \log \frac{en}{y}$ is an increasing function for $y \in [0, n]$, we have

$$\log \mathrm{Syn} \le \sum_{x:\ \text{non-free}} m_x \log \frac{ek}{m_x} \le \sum_{x:\ \text{non-free}} m_x \log \frac{en}{m_x} \le \sum_{x \in [n]} m'_x \log \frac{en}{m'_x} \le n \log e + n \,\mathrm{HMulti},$$

with the convention "$0 \log \frac{en}{0} = 0$". This shows $\mathrm{Syn} \preceq \mathrm{HMulti}$.

Finally, observe that $\sum_{x:\text{ non-free}} m_x \leq \text{Block}$ and $k \leq \text{Block}$. Hence

$$\log \text{Syn} \leq \sum_{x:\text{ non-free}} m_x \log \frac{ek}{m_x} \leq \sum_{x:\text{ non-free}} m_x \log(ek) \leq n \log e + \text{Block} \log \text{Block},$$

which shows $\text{Syn} \preceq \text{Block}$. ◀

▶ **Lemma 48.** *If X contains no duplicates, then* $\text{loc} \preceq \text{Syn}$

If X contains duplicates, then $\text{loc} \not\preceq \text{Syn}$ since $\text{Syn} \preceq \text{HMulti}$ but $\text{loc} \not\preceq \text{HMulti}$.

Proof. We use the notation of Definition 45. If X contains no duplicates, then $m_x = 1$ for every non-free value x. Thus, if n' denotes the number of non-free values, then

$$\log \text{Syn} = \sum_{x:\text{ non-free}} \log \binom{k}{m_x} = n' \log k.$$

Consider a single maximal contiguous non-decreasing subsequence X_i and suppose it is of the form $X[a_i], X[a_i+1], \ldots, X[b_i]$. Let $n'_i = |\{j \in [a_i, b_i] : X[j] \neq X[j-1]+1\}|$ denote the number of non-free values in X_i. For all $x, y > 0$, we have $\log(x) = \log \frac{x}{y} + \log(y) \leq \log \frac{x}{y} + y$. Therefore,

$$\sum_{j=a_i+1}^{b_i} \log(X[j]-X[j-1]) = \sum_{\substack{j=a_i+1 \\ X[j]\text{ non-free}}}^{b_i} \log(X[j]-X[j-1]) \leq \sum_{\substack{j=a_i+1 \\ X[j]\text{non-free}}}^{b_i} (\log k + \frac{X[j]-X[j-1]}{k}) \leq n'_i \log k + \frac{n}{k}.$$

Also, $\log |X[a_i] - X[a_i - 1]| \leq \log n$. Therefore, summing over all X_i yields

$$\log \text{loc} = \sum_{j=2}^{n} \log |X[j] - X[j-1]| \leq k \log n + \sum_{i=1}^{k} (n'_i \log k + \frac{n}{k}) = k \log n + n' \log k + n.$$

We have $k \leq n'$ since $X[a_i]$ is non-free for every i. By Lemma 28 (2), $k \log n \leq 2n + k \log k$. Therefore

$$\log \text{loc} \leq k \log n + n' \log k + n \leq 3n + k \log k + n' \log k \leq 3n + 2n' \log k \in O(n + \log \text{Syn}).$$

This shows $\text{loc} \preceq \text{Syn}$. ◀

E.1 Weighted loc and reg.

To extend Lemma 48 to the case where X contains duplicates, we define a weighted version of loc. Recall that loc roughly corresponds to the dynamic finger bound. Bose, Douïeb, Iacono, and Langerman [9] define a weighted version of the dynamic finger bound (see also [22]), where each value has an arbitrary positive weight. We adapt this definition and define:

▶ **Definition 49.** *For $x \in [n]$, let $w(x)$ denote the number of occurrences of the value x in X. Put $W(x,y) = \sum_{z=\min(x,y)}^{\max(x,y)} w(z)$ and $d_w(x,y) = \frac{W(x,y)}{\min(w(x),w(y))}$. We define*

$$\text{wloc}(X) = \prod_{i=2}^{n} d_w(X[i], X[i-1]) \qquad \textit{with} \quad f_{\text{wloc}} = n + \log \text{wloc},$$

$$\text{wreg}(X) = \prod_{i=2}^{n} \min_{j<i} \left((i-j) \cdot d_w(X[j], X[i])\right) \qquad \textit{with} \quad f_{\text{wreg}} = n + \log \text{wreg}\,.$$

▶ **Lemma 50.** wloc $\preceq$ Syn.

Proof. We use the notation of Definition 45. Our goal is to upper bound the sum

$$\log \mathrm{wloc} = \sum_{j=2}^{n} \log d(X[j], X[j-1]).$$

First, observe that, if $X[j] = X[j-1]$, then $d(X[j], X[j-1]) = 1$. Hence, such indices j have no contribution to $\log \mathrm{wloc}$. Next, suppose that $X[j] \neq X[j-1]$ and that one of the values $X[j], X[j-1]$ is free. No matter which of these values is free, we have $\overline{X[j]} = \overline{X[j-1]} + 1$. This has two consequences. First, this forces $X[j] = X[j-1] + 1$ since $X : [n] \to [n]$ is surjective onto $[\max(X)]$. Thus,

$$d(X[j], X[j-1]) = \frac{X[j-1] + X[j]}{\min(w(X[j-1]), w(X[j]))} \leq w(X[j-1]) + w(X[j]),$$

and we charge $w(X[j-1])$ to the value $X[j-1]$ and $w(X[j])$ to the value $X[j]$. Second, $\overline{X[j]} = \overline{X[j-1]}$ and $X[j] \neq X[j-1]$ means that j is the rightmost occurrence of the value $X[j]$, while $(j-1)$ is the leftmost occurrence of the value $X[j-1]$. Thus, every value gets charged at most twice. Therefore, we have shown

$$\sum_{\substack{j=2 \\ X[j]\neq X[j-1] \\ X[j] \text{ or } X[j-1] \text{ is free}}}^{n} \log d(X[j], X[j-1]) \leq 2n.$$

What remains is to consider the case where $X[i]$ and $X[i-1]$ are both non-free. We follow the proof of Lemma 48. Consider a single maximal contiguous non-decreasing subsequence X_i and suppose it is of the form $X[a_i], X[a_i+1], \ldots, X[b_i]$. For all $x, y > 0$, we have $\log(xy) = \log(x) + \log(y) \leq \log(x) + y$. Let's call this trick *log-splitting*. Therefore,

$$\sum_{\substack{j=a_i+1 \\ X[j]\neq X[j-1] \\ X[j],X[j-1] \text{ non-free}}}^{b_i} \log d(X[j], X[j-1]) \leq \sum_{\substack{j=a_i+1 \\ X[j]\neq X[j-1] \\ X[j],X[j-1] \text{ non-free}}}^{b_i} \left(\log \frac{k}{\min(w(X[j]), w(X[j-1]))} + \frac{W(X[j], X[j-1])}{k} \right).$$

Since X_i is non-decreasing and the sum only considers indices j with $X[j] \neq X[j-1]$, across the whole sum, for each value $x \in [n]$, there are only two terms $W(X[j], X[j-1])$ that include $w(x)$. Since the total weight is n, this shows

$$\sum_{\substack{j=a_i+1 \\ X[j]\neq X[j-1] \\ X[j],X[j-1] \text{ non-free}}}^{b_i} \frac{W(X[j], X[j-1])}{k} \leq 2\frac{n}{k}.$$

Next, we argue that

$$\sum_{\substack{j=a_i+1 \\ X[j]\neq X[j-1] \\ X[i],X[j-1] \text{ non-free}}}^{b_i} \log \frac{k}{\min(w(X[j]), w(X[j-1]))} \leq 2 \sum_{\substack{x \in X_i \\ x \text{ non-free}}} \log \frac{k}{m_x}.$$

Indeed, since X_i is non-decreasing, for each non-free value $x \in X_i$ the term $\frac{k}{w(x)}$ shows up in the left-hand sum at most twice, namely once as $w(X[j])$ and once as $w(X[j-1])$. We have

$w(x) \geq m_x$ for all $x \in X$, and the extra terms on the right are all positive since $m_x \leq k$ for all $x \in X$. Thus, all together, we have shown

$$\sum_{\substack{j=a_i+1 \\ X[j]\neq X[j-1] \\ X[j],\, X[j-1] \text{ non-free}}}^{b_i} \log d(X[j], X[j-1]) \leq 2\frac{n}{k} + 2 \sum_{\substack{x\in X_i \\ x \text{ non-free}}} \log \frac{k}{m_x}. \tag{4}$$

If $i > 1$, then for the first element of each X_i, by log-splitting, we have

$$\begin{aligned} d(X[a_j], X[a_j-1]) &\leq \log \frac{n}{\min(w(X[a_j]), w(X[a_j-1])} \\ &\leq \log \frac{k}{\min(w(X[a_j]), w(X[a_j-1]))} + \frac{n}{k} \\ &\leq \log \frac{k}{\min(m_{X[a_j]}, m_{X[a_j-1]})} + \frac{n}{k} \\ &\leq \log \frac{k}{m_{X[a_j]}} + \log \frac{k}{m_{X[a_j-1]}} + \frac{n}{k}. \end{aligned}$$

We have $X[a_j] \in X_i$ and $X[a_j-1] \in X_{i-1}$. Since $X[a_j-1] > X[a_j]$, neither of the values $X[a_j], X[a_j-1]$ is free, so we can very loosely bound

$$\log \frac{k}{m_{X[a_j]}} + \log \frac{k}{m_{X[a_j-1]}} + \frac{n}{k} \leq \sum_{\substack{x\in X_{i-1} \\ x \text{ non-free}}} \log \frac{k}{m_x} + \sum_{\substack{x\in X_i \\ x \text{ non-free}}} \log \frac{k}{m_x} + \frac{n}{k}.$$

Together with (4), this concludes the analysis of a single X_i. We sum over all X_i and obtain

$$\sum_{\substack{j=2 \\ X[j]\neq X[j-1] \\ X[j],\, X[j-1] \text{ non-free}}}^{n} \log d(X[j], X[j-1]) \leq \sum_{i=1}^{k} \left(4 \sum_{\substack{x\in X_i \\ x \text{ non-free}}} \log \frac{k}{m_x} + 3\frac{n}{k}\right) \leq 4 \log \mathrm{Syn} + 3n.$$

Finally, we put everything together.

$$\begin{aligned} \sum_{j=2}^{n} \log d(X[j], X[j-1]) &= \sum_{\substack{j=2 \\ X[j]\neq X[j-1] \\ X[j] \text{ or } X[j-1] \text{ is free}}}^{n} \log d(X[j], X[j-1]) + \sum_{\substack{i=2 \\ X[j]\neq X[j-1] \\ X[j],\, X[j-1] \text{ non-free}}}^{n} \log d(X[j], X[j-1]) \\ &\leq 2n + (4 \log \mathrm{Syn} + 3n) \in O(n + \log \mathrm{Syn}) \end{aligned}$$

This shows wloc $\preceq$ Syn. ◀

We remark that Lemma 8 generalises to wreg $\preceq$ wloc. We can also show:

▶ **Lemma 51.** wreg $\preceq$ reg

Proof. By log-splitting, for any values $x, y \in [n]$, we have

$$\begin{aligned} \log d_w(x,y) = \log \frac{W(x,y)}{\min(w(x), w(y))} &\leq 2\log(1+|x-y|) + \frac{W(x,y)}{(1+|x-y|)^2 \min(w(x), w(y))} \\ &\leq 2\log(1+|x-y|) + \frac{W(x,y)}{(1+|x-y|)^2 w(x)}. \end{aligned}$$

DANIEL SAYS: This doesn't work, if $x(y) \ll w(x)$ then you have to charge this to the index-distance. If a heavy item has a lot of fingers to a light item, then the index distance is large.

Therefore, for any finger function f, we have

$$\begin{aligned}
\log \mathrm{wreg} &\le \sum_{i=2}^{n} \big(\log(i-f(i)) + \log d_w(x,y))\big) \\
&\le \sum_{i=2}^{n} \big(\log(i-f(i)) + 2\log(1+|X[i]-X[f(i)]|)\big) + \sum_{i=2}^{n} \frac{W(X[i],X[f(i)])}{(1+|X[i]-X[f(i)]|)^2 w(X[i])} \\
&\le 2\log \mathrm{reg} + \underbrace{\sum_{i=2}^{n} \sum_{z=\min(X[i],X[f(i)])}^{\max(X[i],X[f(i)])} \frac{w(z)}{(1+|X[i]-X[f(i)]|)^2 w(X[i]))}}_{=:A_f}.
\end{aligned}$$

In particular, if we can show $A_f \in O(n)$, then $\log \mathrm{wreg} \in O(n + \log \mathrm{reg})$, which shows $\mathrm{wreg} \prec \mathrm{reg}$. To upper bound A_f, observe that

$$\begin{aligned}
A_f &\le \sum_{i=2}^{n} \sum_{z=\min(X[i],X[f(i)])}^{\max(X[i],X[f(i)])} \frac{w(z)}{(1+|X[i]-z|)^2 w(X[i]))} \\
&\le \sum_{i=1}^{n} \sum_{z\in[n]} \frac{w(z)}{(1+|X[i]-z|)^2 w(X[i]))} \\
&\le \sum_{z\in[n]} \sum_{x\in[n]} \frac{w(z)\,w(x)}{1+|x-z|^2\,w(x)} \\
&\le \sum_{z\in[n]} w(z) \sum_{i=1}^{\infty} \frac{2}{1+i^2}
\end{aligned}$$

The infinite series converges, and the total weight is n, so this shows $A_f \in O(n)$. ◀

We are not aware of any sorting algorithm that is wreg- or wloc-adaptive.

F Open Problems

F.1 HSMS and HSUS-adaptive sorting

In this paper, we have described adaptive algorithms for most of the strongest measures in **??**: insertion sort variants for Reg, Hist, and Loc; heapsort variants for * and *; a conditional heapsort construction for * assuming an adjacent-working-set BST; and SkeletonSort for reg. While a reg-adaptive sorting algorithm is automatically HSMS-adaptive and HSUS-adaptive, we are not aware of any HSMS-adaptive or HSUS-adaptive sorting algorithm that does not go via reg (or Reg$/^*$). Since HSMS, HSUS are weaker notions than reg, it should be easier to design an adaptive algorithm for them, but we were unable to do so.

F.2 loc-adaptive heapsort and the mystery of *

Both for reg and hist, we obtain an adaptive heapsort algorithm via the stronger measures * and *. However, our attempt at doing so for loc failed as $^* \not\preceq$ loc. This suggests that * is the "wrong" definition. Both heap definitions are motivated by suffix ranks. However, after dualisation, * dominates hist, whereas * does not dominate the corresponding full-rank measure loc.

Since $^* \not\preceq$ loc, our "adjacent working set property" does not yield a loc-adaptive heapsort algorithm. It would be interesting to find an adaptive heap property that does achieve this, and study the resulting measure of presortedness, which is slightly stronger than loc.

References

1. Noga Alon and Alon Orlitsky. Source coding and graph entropies. *IEEE Transactions on Information Theory*, 42(5):1329–1339, 1996. doi:10.1109/18.532875.
2. Tom Altman and Yoshihide Igarashi. Roughly sorting: Sequential and parallel approach. *Journal of Information Processing*, 12(2):154–158, 1989.
3. Mihai Bădoiu, Richard Cole, Erik D. Demaine, and John Iacono. A unified access bound on comparison-based dynamic dictionaries. *Theoretical Computer Science*, 382(2):86–96, 2007. doi:10.1016/j.tcs.2007.03.002.
4. Xingjian Bai and Christian Coester. Sorting with predictions. In *Advances in Neural Information Processing Systems*, volume 36, pages 26563–26584, 2023. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/544696ef4847c903376ed6ec58f3a703-Abstract-Conference.html.
5. Jérémy Barbay, Johannes Fischer, and Gonzalo Navarro. LRM-trees: Compressed indices, adaptive sorting, and compressed permutations. *Theoretical Computer Science*, 459:26–41, 2012. doi:10.1016/j.tcs.2012.08.010.
6. Jérémy Barbay and Gonzalo Navarro. On compressing permutations and adaptive sorting. *Theoretical Computer Science*, 513:109–123, 2013. doi:10.1016/j.tcs.2013.10.019.
7. Jérémy Barbay, Carlos Ochoa, and Srinivasa Rao Satti. Synergistic solutions on multisets. In *28th Annual Symposium on Combinatorial Pattern Matching (CPM 2017)*, volume 78 of *Leibniz International Proceedings in Informatics*, pages 31:1–31:14. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017. doi:10.4230/LIPIcs.CPM.2017.31.
8. Guy E. Blelloch and Magdalen Dobson. The geometry of tree-based sorting. In *50th International Colloquium on Automata, Languages, and Programming (ICALP 2023)*, volume 261 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.ICALP.2023.26.
9. Prosenjit Bose, Karim Douïeb, John Iacono, and Stefan Langerman. The power and limitations of static binary search trees with lazy finger. *Algorithmica*, 76(4):1264–1275, 2016.
10. Svante Carlsson, Christos Levcopoulos, and Ola Petersson. Sublinear merging and natural mergesort. *Algorithmica*, 9(6):629–648, 1993. Preliminary version appeared in ISAAC 1990. doi:10.1007/BF01190160.
11. Richard Cole. On the dynamic finger conjecture for splay trees. part II: The proof. *SIAM Journal on Computing*, 30(1):44–85, 2000. doi:10.1137/S009753979732699X.
12. Richard Cole, Bud Mishra, Jeanette Schmidt, and Alan Siegel. On the dynamic finger conjecture for splay trees. part I: Splay sorting $\log n$-block sequences. *SIAM Journal on Computing*, 30(1):1–43, 2000. doi:10.1137/S0097539797326988.
13. Curtis R. Cook and Do Jin Kim. Best sorting algorithm for nearly sorted lists. *Communications of the ACM*, 23(11):620–624, 1980. doi:10.1145/359024.359026.
14. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
15. Erik D. Demaine, Dion Harmon, John Iacono, Daniel Kane, and Mihai Pătraşcu. The geometry of binary search trees. In *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2009)*, pages 496–505, 2009. doi:10.1137/1.9781611973068.55.
16. Erik D. Demaine, John Iacono, Stefan Langerman, and Özgür Özkan. Combining binary search trees. In *Automata, Languages, and Programming (ICALP 2013)*, pages 388–399, 2013. doi:10.1007/978-3-642-39206-1_33.

17 Jonathan C. Derryberry and Daniel D. Sleator. Skip-splay: Toward achieving the unified bound in the bst model. In *Algorithms and Data Structures (WADS 2009)*, volume 5664 of *Lecture Notes in Computer Science*, pages 194–205, 2009. doi:10.1007/978-3-642-03367-4_18.

18 Persi Diaconis and Ronald L. Graham. Spearman's footrule as a measure of disarray. *Journal of the Royal Statistical Society. Series B (Methodological)*, 39(2):262–268, 1977. doi:10.1111/j.2517-6161.1977.tb01624.x.

19 Vladimir Estivill-Castro and Derick Wood. A new measure of presortedness. *Information and Computation*, 83(1):111–119, 1989. doi:10.1016/0890-5401(89)90050-3.

20 Elahe Ghasemi, Vincent Jugé, and Ghazal Khalighinejad. Galloping in fast-growth natural merge sorts. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 68:1–68:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ICALP.2022.68.

21 John Iacono. Improved upper bounds for pairing heaps. In *Algorithm Theory – SWAT 2000*, volume 1851 of *Lecture Notes in Computer Science*, pages 32–45. Springer, 2000. doi:10.1007/3-540-44985-X_5.

22 John Iacono and Stefan Langerman. *Weighted dynamic finger in binary search trees*, pages 672–691. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611974331.ch49, doi:10.1137/1.9781611974331.ch49.

23 Jyrki Katajainen, Christos Levcopoulos, and Ola Petersson. Local insertion sort revisited. In *Optimal Algorithms*, volume 401 of *Lecture Notes in Computer Science*, pages 239–253. Springer, 1989. doi:10.1007/3-540-51859-2_19.

24 Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition, 1998.

25 Christos Levcopoulos and Ola Petersson. Adaptive heapsort. *Journal of Algorithms*, 14(3):395–413, 1993. doi:10.1006/jagm.1993.1021.

26 Christos Levcopoulos and Ola Petersson. Sorting shuffled monotone sequences. *Information and Computation*, 112(1):37–50, 1994. doi:10.1006/inco.1994.1050.

27 Heikki Mannila. Measures of presortedness and optimal sorting algorithms. *IEEE Transactions on Computers*, C-34(4):318–325, 1985. doi:10.1109/TC.1985.5009382.

28 Peter McIlroy. Optimistic sorting and information theoretic complexity. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 467–474. Society for Industrial and Applied Mathematics, 1993. URL: https://dl.acm.org/doi/10.5555/313559.313859.

29 Alistair Moffat and Ola Petersson. Historical searching and sorting. In *Algorithms: Second International Symposium, ISA '91*, volume 557 of *Lecture Notes in Computer Science*, pages 263–272. Springer, 1991. doi:10.1007/3-540-54945-5_70.

30 J. Ian Munro and Philip M. Spira. Sorting and searching in multisets. *SIAM Journal on Computing*, 5(1):1–8, 1976. doi:10.1137/0205001.

31 Ola Petersson and Alistair Moffat. A framework for adaptive sorting. *Discrete Applied Mathematics*, 59(2):153–179, 1995. doi:10.1016/0166-218X(93)E0160-Z.

32 Daniel Rutschmann. Sorting under partial information with optimal preprocessing time via unified bound heaps. In *Proceedings of the 67th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2026. To appear.

33 Raimund Seidel and Cecilia R Aragon. Randomized search trees. *Algorithmica*, 16(4):464–497, 1996.

34 Steven S. Skiena. Encroaching lists as a measure of presortedness. *BIT*, 28(4):775–784, 1988. doi:10.1007/BF01954897.

35 Daniel Dominic Sleator and Robert Endre Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985. doi:10.1145/3828.3835.

36 Klaus W. Wagner. Monotonic coverings of finite sets. *Elektronische Informationsverarbeitung und Kybernetik*, 20(12):633–639, 1984.